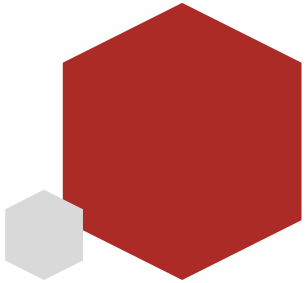
Flutter从入门到实战

黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



Flutter综合案例

案例需求和规划

案例效果展示

首页、登录页、我的页面

案例开发规划

1. 首先完成纯web端功能
2. 开发过程中串联之前的基础知识、扩展插件、弹层、provider、getx等技术
3. 开始多端适配、安卓 =》 ios =》鸿蒙适配等鸿蒙化技术
4. 课程完结
5. 小知识点采用碎片化更新

创建项目–基本目录使用git管理

1. 首选创建适配web平台项目

```
flutter create --platforms web hm_shop
```

2. 创建基本目录结构

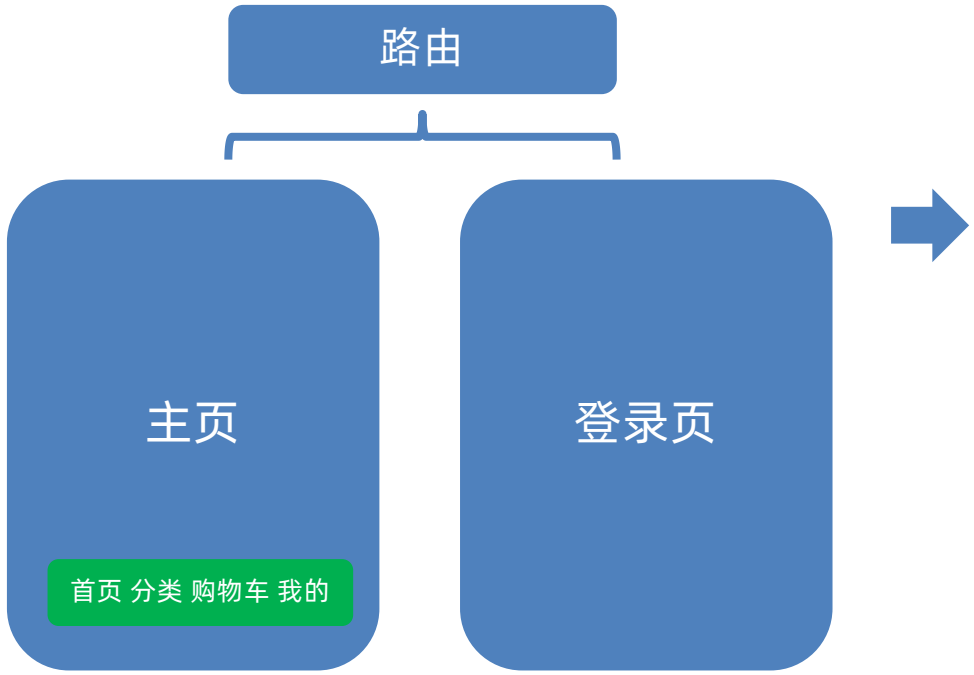
```
-lib
  -api            #存放请求
  -assets         #存放资源
  -components     #存放公共组件
  -contants       #存放常量文件
  -viewmodels     #存放类型文件
  -pages          #存放页面
  -routes         #存放路由配置
  -stores         #存放全局状态组件
  -utils          #存放工具类
  -main.dart      #入口
```

3. 创建本地和远程仓库并提交

```
git init # 初始化仓库
git add . # 提交本地仓库到暂存区
git commit -m “初始化仓库” # 提交本地仓库
git remote add origin master <远程仓库地址> # 添加远程仓库
git push -u origin master # 推送主分支到远程
```

搭建基础路由和组件

基础路由和页面组件



1.新建路由配置文件-routes/index.dart

```dart
Widget getRootWidget() {
  return MaterialApp(initialRoute: "/", routes: getRootRoutes());
}

Map<String, Widget Function(BuildContext)> getRootRoutes() {
  return {"/": (context) => MainPage(), "/login": (context) => LoginPage()};
}
试试 ⌘U 与 AI 聊天，⌘I 与 AI 一起编写代码。
```

2.入口处运行-main.dart

```dart
Run | Debug | Profile
void main() {
  runApp(getRootWidget());
}
```

搭建基础路由和组件

3.新建MainPage和LoginPage

4.先写个基本结构

```
class _MainPageState extends State<MainPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("主页")),
      body: Center(child: Text("主页")),
    ); // Scaffold
  }
}
```

主页Tab栏

SafeArea（避开安全区组件）

IndexedStack（堆叠组件，根据索引显示对应组件）

BottomNavigationBar（上图下字组件，可切换索引）

主页Tab栏–代码实现

1.将素材拷贝到项目lib/assets目录下

2.配置pubspec.yaml文件的资源路径

```
section, like this:
assets:
  - lib/assets/
#   - images/a_dot_ham.jpeg
```

3.定义底部循环项数据

```
final List<Map<String, String>> _tabList = [
  {
    "icon": 'lib/assets/ic_public_home_normal.png',
    "active_icon": 'lib/assets/ic_public_home_active.png',
    "name": '首页',
  },
  {
    "icon": 'lib/assets/ic_public_pro_normal.png',
    "active_icon": 'lib/assets/ic_public_pro_active.png',
    "name": '分类',
  },
  {
    "icon": 'lib/assets/ic_public_cart_normal.png',
    "active_icon": 'lib/assets/ic_public_cart_active.png',
    "name": '购物车',
  },
  {
    "icon": 'lib/assets/ic_public_my_normal.png',
    "active_icon": 'lib/assets/ic_public_my_active.png',
    "name": '我的',
  },
];
```

主页Tab栏–代码实现

4.使用函数循环生成循环项

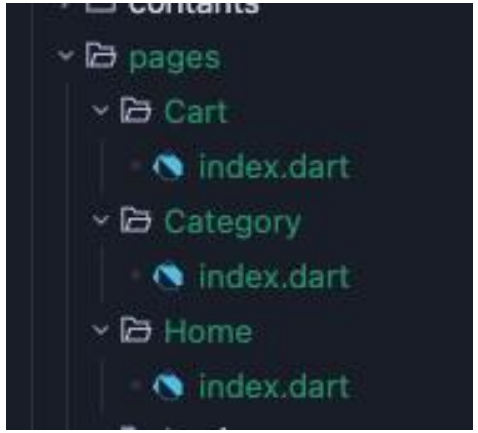
```
List<BottomNavigationBarItem> _getTabBarWidget() {
  return List.generate(_tabList.length, (index) {
    return BottomNavigationBarItem(
      icon: Image.asset(_tabList[index]["icon"]!, width: 30, height: 30),
      activeIcon: Image.asset(
        _tabList[index]["active_icon"]!,
        width: 30,
        height: 30,
      ), // Image.asset
      label: _tabList[index]["name"],
    ); // BottomNavigationBarItem
  }); // List.generate
}
```

5.配置BottomNavigationBar

```
bottomNavigationBar: BottomNavigationBar(
  showUnselectedLabels: true,
  useLegacyColorScheme: false,
  selectedItemColor: Colors.black,
  unselectedLabelStyle: TextStyle(color: Colors.black),
  onTap: (index) {
    setState(() {
      _currentIndex = index;
    });      You, 14分钟前 • Uncommitted changes
  },
  currentIndex: _currentIndex,
  items: _getTabBarWidget(),
), // BottomNavigationBar
```

主页Tab栏–代码实现

6.创建四个组件视图-首页-分类-购物车-我的



7.使用IndexedStack放置

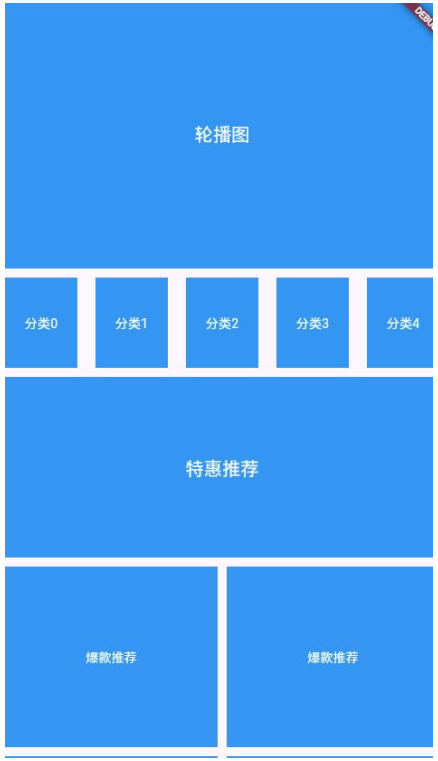
```dart
int _currentIndex = 0;
List<Widget> _getShowWidget() {
  return [HomeView(), CategoryView(), CartView(), MineView()];
}
```

```dart
body: SafeArea(
  child: IndexedStack(index: _currentIndex, children: _getShowWidget()),
), // SafeArea
```

首页布局实现

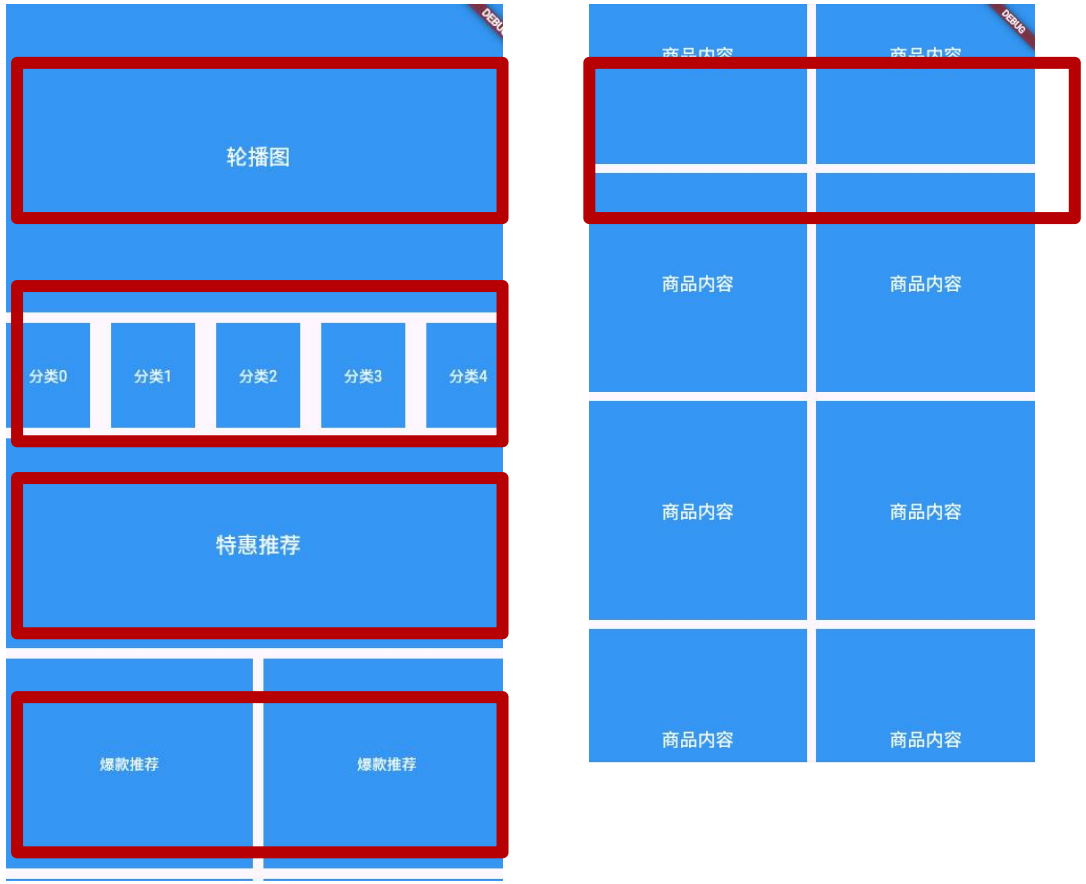

12:29
女神新包
新品限时八折
特惠推荐
年终大促
限时买就送
¥787
¥1809
¥1213
爆款推荐
最受欢迎
¥116
¥1213
一站买全
精心优选
¥1019
¥1809


新款LOGO印花时尚学步叫叫鞋
¥259
趣味小恐龙绣花，男童灯芯绒衬衫
¥89
跟灯按弦超好玩碳纤维智能尤克里里
¥599
亚洲头围城市运动通勤电动车头盔
¥158


轮播图
分类0
分类1
分类2
分类3
分类4
特惠推荐
爆款推荐
爆款推荐


商品内容
商品内容
商品内容
商品内容
商品内容
商品内容
商品内容
商品内容

首页布局–代码实现



CustomScrollView

- HmSlider(录播图)
- HmCategory（分类）
- HmSuggestion（推荐）
- HmHot（爆款）
- HmMoreList（无限滚动）

首页布局–代码实现

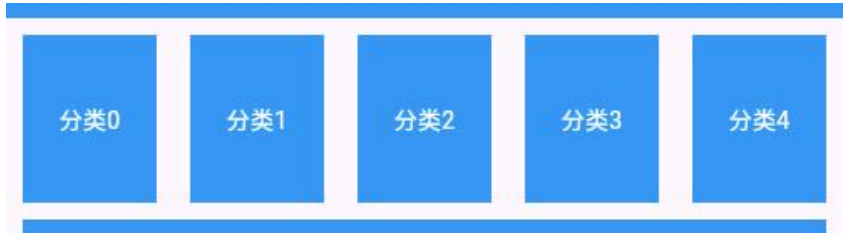
1.创建5个组件-components/Home

2.轮播图组件


```dart
class HmSlider extends StatefulWidget {
  HmSlider({Key? key}) : super(key: key);

  @override
  _HmSliderState createState() => _HmSliderState();
}

class _HmSliderState extends State<HmSlider> {
  @override
  Widget build(BuildContext context) {
    return Container(
          alignment: Alignment.center,
          color: Colors.blue,
          height: 300,
          child: Text(
            "轮播图",
            style: TextStyle(color: Colors.white, fontSize: 20),
          ), // Text
        ); // Container
  }
}
```

首页布局–代码实现

2.分类组件

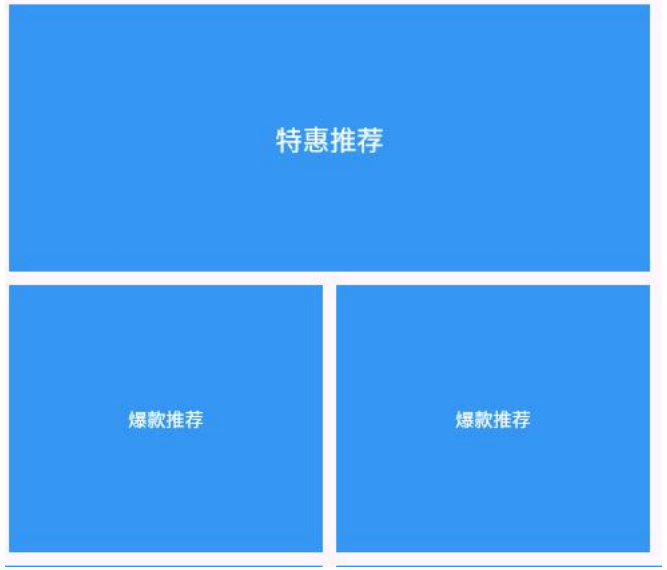


ListView外层要包SizeBox或者Container确定高度

```dart
class _HmCategoryState extends State<HmCategory> {
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 100,
      child: ListView.builder(
        scrollDirection: Axis.horizontal,
        itemCount: 10,
        itemBuilder: (BuildContext context, int index) {
          return Container(
            height: 100,
            width: 80,
            color: Colors.blue,
            margin: EdgeInsets.symmetric(horizontal: 10),
            alignment: Alignment.center,
            child: Text("分类$index", style: TextStyle(color: Colors.white)),
          ); // Container
        },
      ), // ListView.builder
    ); // SizedBox
  }
}
```

首页布局–代码实现

3.特惠推荐和爆款推荐




```dart
class _HmHotState extends State<HmHot> {
  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.blue,
      alignment: Alignment.center,
      child: Text("爆款推荐", style: TextStyle(color: Colors.
      white)),
    ); // Container
  }
}
```

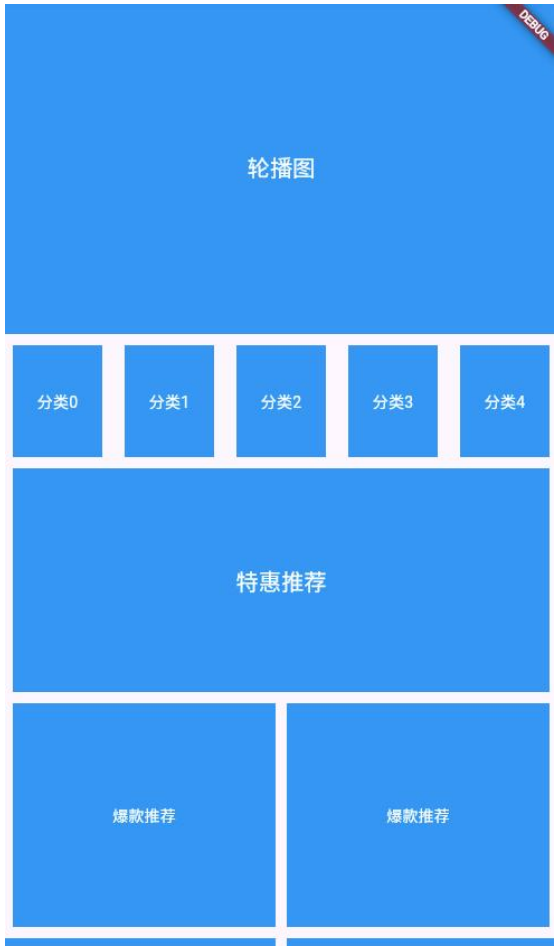
首页布局–代码实现

4.无限滚动列表


```dart
class _HmMoreListState extends State<HmMoreList> {
  @override
  Widget build(BuildContext context) {
    return SliverGrid.builder(
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2,
        mainAxisSpacing: 10,
        crossAxisSpacing: 10,
      ), // SliverGridDelegateWithFixedCrossAxisCount
      itemCount: 100,
      itemBuilder: (BuildContext context, int index) {
        return Container(
          color: Colors.blue,
          alignment: Alignment.center,
          child: Text(
            "商品内容",
            style: TextStyle(color: Colors.white, fontSize: 20),
          ), // Text
        ); // Container
      },
    ); // SliverGrid.builder
  }
}
```

首页布局–代码实现

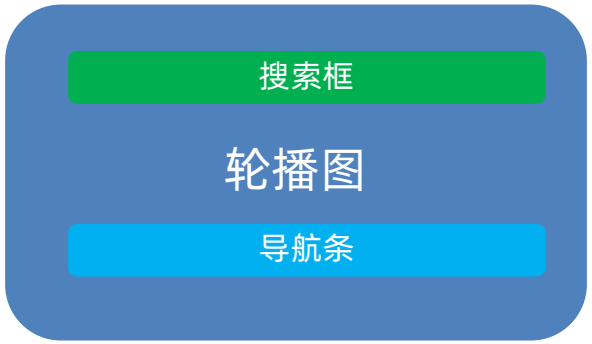
5.在CustomScrollView中组合




```dart
@override
Widget build(BuildContext context) {
  return CustomScrollView(slivers: _getHomeChildren());
}
```

```dart
List<Widget> _getHomeChildren() {
  return [
    SliverToBoxAdapter(child: HmSlider()),
    SliverToBoxAdapter(child: SizedBox(height: 10)),
    SliverToBoxAdapter(child: HmCategory()),
    SliverToBoxAdapter(child: SizedBox(height: 10)),
    SliverToBoxAdapter(child: HmSuggestion()),
    SliverToBoxAdapter(child: SizedBox(height: 10)),
    SliverToBoxAdapter(
      child: Padding(
        padding: EdgeInsets.symmetric(horizontal: 10),
        child: SizedBox(
          height: 200,
          child: Flex(
            direction: Axis.horizontal,
            children: [      You, 44秒钟前 • Uncommitted changes
              Expanded(child: HmHot()),
              SizedBox(width: 10),
              Expanded(child: HmHot()),
            ],
          ), // Flex
        ), // SizedBox
      ), // Padding
    ), // SliverToBoxAdapter
    SliverToBoxAdapter(child: SizedBox(height: 10)),
```

实现轮播图



Stack

轮播图

Positioned

搜索框

导航条

安装轮播图插件

```
flutter pub add carousel_slider
```

准备几个测试图片地址

```
https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/1.jpg
https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/2.png
https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/3.jpg
```

实现轮播图

定义轮播图数据对象类型-viewmodels/home.dart

```dart
class BannerItem {
  String id;
  String imgUrl;
  BannerItem({required this.id, required this.imgUrl});
}
```

在HomeView组件中定义List数据

```dart
final List<BannerItem> _bannerList = [
  BannerItem(
    id: "1",
    imgUrl: "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/1.jpg",
  ),
  BannerItem(
    id: "2",
    imgUrl: "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/2.png",
  ),
  BannerItem(
    id: "3",
    imgUrl: "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meituan/3.jpg",
  ),
];
```

HmSlider组件接收数据

```dart
You, 11分钟前 | 1 author (You)
class HmSlider extends StatefulWidget {
  final List<BannerItem> bannerList;
  HmSlider({Key? key, required this.bannerList}) : super(key: key);

  @override
  _HmSliderState createState() => _HmSliderState();
}
      You, 昨天 • 完成首页基础布局
```

实现轮播图

使用Stack包裹CarouselSlider组件

```dart
@override
Widget build(BuildContext context) {
  return Stack(children: [_getSlider()]);
}
```

```dart
final screenWidth = MediaQuery.of(context).size.width;

return CarouselSlider(
  items: List.generate(widget.bannerList.length, (int index) {
    return Image.network(
      widget.bannerList[index].imgUrl,
      fit: BoxFit.cover,
      width: screenWidth,
    ); // Image.network
  }), // List.generate
  options: CarouselOptions(
    autoPlay: true,
    autoPlayCurve: Curves.linear,
    height: 300,
    viewportFraction: 1,
  ), // CarouselOptions      You, 13分钟前 · Uncommitted changes
); // CarouselSlider
}
```

知识点：整体屏幕的宽度可以通过MediaQuery.of(context).size.width获取

实现轮播图–搜索和导航条

搜索Widget

```
Widget _getSearch() {
  return Positioned(
    top: 10,
    left: 0,
    right: 0,
    child: Padding(
      padding: EdgeInsets.all(10),
      child: Container(
        alignment: Alignment.centerLeft,
        padding: EdgeInsets.symmetric(horizontal: 40),
        height: 50,
        decoration: BoxDecoration(
          color: const Color.fromRGBO(0, 0, 0, 0.4),
          borderRadius: BorderRadius.circular(25),      You, 44分钟前 • Unco
        ), // BoxDecoration
        child: Text(
          "搜索...",
          style: TextStyle(color: Colors.white, fontSize: 16),
        ), // Text
      ), // Container
    ), // Padding
  ); // Positioned
}
```

实现轮播图–搜索和导航条

导航条Widget

```
Widget _getDots() {
  return Positioned(      You, 44分钟前 • Uncommitted changes
    bottom: 10,
    left: 0,
    right: 0,
    child: SizedBox(
      height: 40,
      width: double.infinity,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.center,
```

```
children: List.generate(widget.bannerList.length, (int index) {
  return GestureDetector(
    onTap: () {
      _controller.jumpToPage(index);
    },
    child: AnimatedContainer(
      duration: Duration(milliseconds: 300),
      width: index == _currentIndex ? 40 : 20,
      height: 6,
      margin: EdgeInsets.symmetric(horizontal: 4),
      decoration: BoxDecoration(
        color: index == _currentIndex
            ? Colors.white
            : Color.fromRGBO(0, 0, 0, 0.4),
        borderRadius: BorderRadius.circular(3),
      ), // BoxDecoration
    ), // AnimatedContainer
  ); // GestureDetector
}), // List.generate
```

获取轮播图数据

步骤

1. 安装dio
2.定义常量数据、基础地址、超时时间、业务状态、请求地址
3.封装网络请求工具，基础地址，拦截器
4.请求工具进一步解构，处理http状态和业务状态
5. 类工厂转化动态类型到对象类型
6.封装请求API调用工厂函数
7.初始化数据更新状态

接口基础地址：https://meikou-api.itheima.net

轮播图地址(get)：https://meikou-api.itheima.net/home/banner


```
{
    "code": "1",
    "msg": "操作成功",
    "result": [
        {
            "id": "45",
            "imgUrl": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/banner/xinnian_sj.png",
            "hrefUrl": "/category/1181622001",
            "type": "1"
        },
        {
            "id": "42",
            "imgUrl": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/banner/nuandong_sj.png",
            "hrefUrl": "/category/1181622006",
            "type": "1"
        },
        {
            "id": "48",
            "imgUrl": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/banner/nvshen_sj.png",
            "hrefUrl": "/category/1181622003",
            "type": "1"
        }
    ]
}
```

获取轮播图数据

1.安装dio插件

```
flutter pub add dio
```

2.定义常量数据

```dart
class GlobalConstants {
  static const String BASE_URL = "https://meikou-api.itheima.net"; // 基础地址
  static const String SUCCESS_CODE = "1"; // 成功状态码
  static const int TIME_OUT = 10; // 超时时间 s单位
}

class HttpContants {
  static const String BANNER_LIST = "/home/banner";
}
```

获取轮播图数据

3.封装网络请求工具-进一步解构

```dart
class DioRequest {
  final _dio = Dio();
  DioRequest() {
    _dio.options
      ..baseUrl = GlobalConstants.BASE_URL
      ..connectTimeout = Duration(seconds: GlobalConstants.TIME_OUT)
      ..receiveTimeout = Duration(seconds: GlobalConstants.TIME_OUT)
      ..sendTimeout = Duration(seconds: GlobalConstants.TIME_OUT);
    _addInterceptor();
  }
```

1.声明dio请求对象、 设置基础地址、 超时时间

```dart
  Future<dynamic> get(String url, {Map<String, dynamic>? params}) {
    return _handleResponse(_dio.get(url, queryParameters: params));
  }

  Future<dynamic> _handleResponse(Future<Response<dynamic>> task) async {
    try {
      dynamic res = await task;
      final data = res.data as Map<String, dynamic>;
      if (data["code"] == GlobalConstants.SUCCESS_CODE) {
        return data["result"] as dynamic;
      }
      throw Exception("加载数据异常");
    } catch (e) {
      throw Exception("加载数据异常");
    }
  }
```

3.封装get方法、判断业务状态码、进一步解构

```dart
_addInterceptor() {
  _dio.interceptors.add(
    InterceptorsWrapper(
      onRequest: (request, handler) {
        handler.next(request);
      },
      onResponse: (response, handler) {
        if (response.statusCode! >= 200 && response.statusCode! < 300) {
          handler.next(response);
          return;
        }
        handler.reject(DioException(requestOptions: response.requestOptions));
      },
      onError: (error, handler) {
        handler.reject(error);
      },
    ), // InterceptorsWrapper
  );
}
```

2.请求拦截器、响应拦截器、错误拦截器

获取轮播图数据

4.类工厂转化类型

```dart
You, 2小时前 | 1 author (You)
class BannerItem {
  String id;
  String imgUrl;
  BannerItem({required this.id, required this.imgUrl});
  //  go
  //  从json中解析数据
  factory BannerItem.fromJson(Map<String, dynamic> json) {
    return BannerItem(
      id: json["id"] as String,
      imgUrl: json["imgUrl"] as String,
    );      You, 2小时前 • Uncommitted changes
  }
}
```

5.封装请求API

```dart
import 'package:hm_shop/contants/index.dart';
import 'package:hm_shop/utils/DioRequest.dart';
import 'package:hm_shop/viewmodels/home.dart';

// Future<List<BannerItem>> getBannerListAPI = () =>

// 获取轮播图列表
Future<List<BannerItem>> getBannerListAPI() async {
  return (await dioRequest.get(HttpContants.BANNER_LIST) as List<dynamic>)
      .map((e) => BannerItem.fromJson(e as Map<String, dynamic>))
      .toList();
}
```

factory可以用来创建构造函数,它能控制对象的创建方式

获取轮播图数据

6.初始化获取数据

```dart
@override
void initState() {
  // TODO: implement initState
  super.initState();
  _getBannerList();
}

void _getBannerList() async {
  _bannerList = await getBannerListAPI();
  setState(() {});
}      You, 2小时前 • Uncommitted changes
```

分类数据获取并渲染(AI)

步骤

1. 定义常量请求地址
2.定义数据类型和工厂转化函数
3.封装API请求
4.初始化获取数据
5.传递数据到子组件
6.子组件完成渲染视图

分类地址(get)：/home/category/head


```
"code": "1",
"msg": "操作成功",
"result": [
    {
        "id": "1181622001",
        "name": "气质女装",
        "picture": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/c1/qznz.png",
        "children": [
            {
                "id": "1191110001",
                "name": "半裙",
                "picture": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/c2/qznz_bq.png?quality=95&imageView",
                "children": null,
                "goods": null
            },
            {
                "id": "1191110002",
                "name": "衬衫",
                "picture": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/c2/qznz_cs.png?quality=95&imageView",
                "children": null,
                "goods": null
            },
            {
                "id": "1191110022",
                "name": "T恤",
                "picture": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/c2/qznz_tx.png?quality=95&imageView",
                "children": null,
                "goods": null
            },
            {
                "id": "1191110023",
                "name": "针织衫",
                "picture": "https://yjy-teach-oss.oss-cn-beijing.aliyuncs.com/meikou/c2/qznz_zzs.png?quality=95&imageView",
```

分类数据获取并渲染(AI)

1.打开Trae编辑器Tab提示

2.定义常量请求地址

```
You, 7小时前 | 1 author (You)
class HttpConstants {
  static const String BANNER_LIST = "/home/banner";
  static const String CATEGORY_LIST = "/home/category/head";
}
试试 ⌘ U 与 AI 聊天, ⌘ I 与 AI 一起编写代码。
```

分类数据获取并渲染(AI)

3.粘贴json数据使用TraeAI(Tab)生成对象类型及工厂转化

```dart
class CategoryItem {
  String id;
  String name;
  String picture;
  List<CategoryItem>? children;
  CategoryItem({
    required this.id,
    required this.name,
    required this.picture,
    this.children,
  });
  // 扩展一个工厂函数 一般用factory来声明 一般用来创建实例对象
  factory CategoryItem.formJSON(Map<String, dynamic> json) {
    // 必须返回一个CategoryItem对象
    return CategoryItem(
      id: json["id"] ?? "",      You, 7小时前 • Uncommitted changes
      name: json["name"] ?? "",
      picture: json["picture"] ?? "",
      children: json["children"] == null
          ? null
          : (json["children"] as List)
                .map((e) => CategoryItem.formJSON(e))
                .toList(),
    ); // CategoryItem
  }
```

分类数据获取并渲染(AI)

4.使用TraeAI(Tab)生成封装AI方法

```dart
//  获取分类数据
Future<List<CategoryItem>> getCategoryListAPI() async {
  // 返回请求
  return ((await dioRequest.get(HttpConstants.CATEGORY_LIST)) as List).map((
    item,
  ) {
    return CategoryItem.formJSON(item as Map<String, dynamic>);
  }).toList();
}
```

分类数据获取并渲染(AI)

5.使用TraeAI初始化获取数据并赋值

```
List<CategoryItem> _categoryList = [];
// 获取滚动容器的内容
```

```
@override
void initState() {
  // TODO: implement initState
  super.initState();
  _getBannderList();
  _getCategoryList();
}

void _getBannderList() async {
  _bannerList = await getBannerListAPI();
  setState(() {});
}

void _getCategoryList() async {
  _categoryList = await getCategoryListAPI();
  setState(() {});
}
```

分类数据获取并渲染(AI)

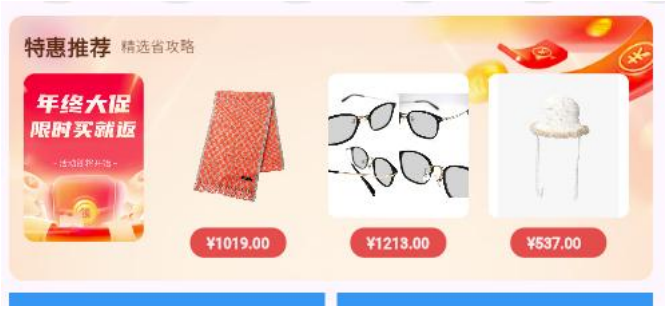
6.使用TraeAI父传子并渲染内容(手动微调)

```
SliverToBoxAdapter(
  child: Hmcategory(categoryList: _categoryList),
), // 分类组件 // SliverToBoxAdapter
SliverToBoxAdapter(child: SizedBox(height: 10)),
```

```
You, 2小时前 | 1 author (You)
class Hmcategory extends StatefulWidget {
  Hmcategory({Key? key, required this.categoryList}) : super(key: key);
  final List<CategoryItem> categoryList;

  @override
  _HmcategoryState createState() => _HmcategoryState();
}
```


```
scrollDirection: Axis.horizontal,
itemCount: widget.categoryList.length,
itemBuilder: (BuildContext context, int index) {
  final item = widget.categoryList[index];
  return Container(
    alignment: Alignment.center,
    width: 80,
    height: 100,
    decoration: BoxDecoration(
      color: const Color.fromARGB(255, 228, 230, 231),
      borderRadius: BorderRadius.circular(40),      You, 2小时前 • Uncommitte
    ), // BoxDecoration
    margin: EdgeInsets.symmetric(horizontal: 10),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Image.network(item.picture, width: 50, height: 50),
        Text(
          item.name,
          style: TextStyle(color: Colors.black, fontSize: 12),
        ), // Text
      ],
    ), // Column
```

获取特惠推荐渲染(AI + 手动)



说明

数据结构较为复杂-普通TabAI难以生成-采用协作模式

UI结构手动编码模式

步骤

1.请求地址常量(AI)

2.根据json转化对象模型(AI协作)

3.封装API请求(AI + 修正)

4.初始化获取数据（AI）

5.传递数据到子组件（AI）

6.拷贝图片资源/取3条子选项进行渲染(手动实现)

特惠推荐(get)：/hot/preference

获取特惠推荐渲染(AI + 手动)

1.定义常量请求地址

```
You, 14小时前 | 1 author (You)
class HttpConstants {
  static const String BANNER_LIST = "/home/banner";
  static const String CATEGORY_LIST = "/home/category/head"; // 分类列表
  static const String HOT_PRODUCT_LIST = "/hot/preference"; // 热门商品列表
}
```

2.准备json文件拷贝到项目中，使用协助模式生成类型

获取特惠推荐渲染(AI + 手动)

特惠推荐数据结构

```dart
class SpecialRecommendResult {
  String id;
  String title;
  List<SubType> subTypes;
  SpecialRecommendResult({
    required this.id,
    required this.title,
    required this.subTypes,
  });
  factory SpecialRecommendResult.formJSON(Map<String, dynamic> json) {
    return SpecialRecommendResult(
      id: json["id"]?.toString() ?? "",
      title: json["title"]?.toString() ?? "",
      subTypes: (json["subTypes"] as List? ?? [])
          .map((item) => SubType.formJSON(item as Map<String, dynamic>))
          .toList(),
    ); // SpecialRecommendResult
  }
}
```

```dart
class SubType {
  String id;
  String title;
  GoodsItems goodsItems;
  SubType({required this.id, required this.title, required this.goodsItems});
  factory SubType.formJSON(Map<String, dynamic> json) {
    return SubType(
      id: json['id'] ?? '',
      title: json['title'] ?? '',
      goodsItems: GoodsItems.formJSON(json['goodsItems'] ?? {}),
    ); // SubType
  }
}
```

```dart
class GoodsItems {
  int counts;
  int pageSize;
  int pages;
  int page;
  List<GoodsItem> items;
  GoodsItems({
    required this.counts,
    required this.pageSize,
    required this.pages,
    required this.page,
    required this.items,
  });
  factory GoodsItems.formJSON(Map<String, dynamic> json) {
    int _toInt(dynamic v) => v is int ? v : int.tryParse('${v ?? 0}') ?? 0;
    return GoodsItems(
      counts: _toInt(json['counts']),
      pageSize: _toInt(json['pageSize']),
      pages: _toInt(json['pages']),
      page: _toInt(json['page']),
      items: (json['items'] as List? ?? [])
          .map((item) => GoodsItem.formJSON(item as Map<String, dynamic>))
          .toList(),
    ); // GoodsItems
  }
}
```

```dart
class GoodsItem {
  String id;
  String name;
  String? desc;
  String price;
  String picture;
  int orderNum;
  GoodsItem({
    required this.id,
    required this.name,
    this.desc,
    required this.price,
    required this.picture,
    required this.orderNum,
  });
  factory GoodsItem.formJSON(Map<String, dynamic> json) {
    int _toInt(dynamic v) => v is int ? v : int.tryParse('${v ?? 0}') ?? 0;
    return GoodsItem(
      id: json['id'] ?? '',
      name: json['name'] ?? '',
      desc: json['desc'],
      price: json['price'] ?? '',
      picture: json['picture'] ?? '',
      orderNum: _toInt(json['orderNum']),
    );
  }
```

获取特惠推荐渲染(AI + 手动)

3.封装API请求

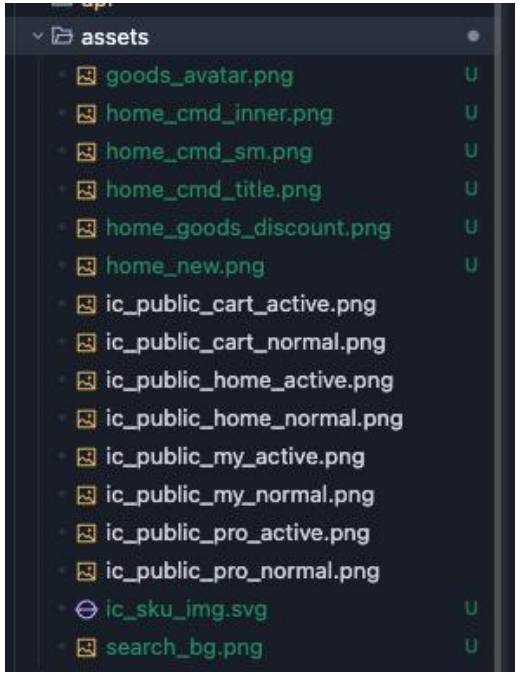
```dart
// 特惠推荐
Future<SpecialRecommendResult> getProductListAPI() async {
  // 返回请求
  return SpecialRecommendResult.formJSON(
    await dioRequest.get(HttpConstants.PRODUCT_LIST),
  );      You, 18小时前 • 实现特惠推荐的渲染 …
}
```

4.初始化获取数据

```dart
@override
void initState() {
  // TODO: implement initState
  super.initState();
  _getBannderList();
  _getCategoryList();      You, 前天 • 使用TraeAI实现获取分类数据并渲染
  _getHotProductList();
}

// 特惠推荐数据模型
PromotionResult _hotProductList = PromotionResult(
  id: '',
  title: '',
  subTypes: [],
);
// 获取热门商品列表
void _getHotProductList() async {
  _hotProductList = await getHotProductListAPI();
  setState(() {});
}
```

获取特惠推荐渲染(AI + 手动)

5.拷贝推荐资源



6.特惠推荐渲染

```
You, 18小时前 | 1 author (You)
class HmSuggestion extends StatefulWidget {
  final SpecialRecommendResult specialRecommendResult;
  HmSuggestion({Key? key, required this.specialRecommendResult})
    : super(key: key);

  @override
  _HmSuggestionState createState() => _HmSuggestionState();
}
```

```
Widget _buildHeader() {
  return Row(
    children: [
      Text(
        "特惠推荐",
        style: TextStyle(
          color: const Color.fromARGB(255, 86, 24, 20),
          fontSize: 18,
          fontWeight: FontWeight.w700,
        ), // TextStyle
      ), // Text
      SizedBox(width: 10),
      Text(
        "精选省攻略",
        style: TextStyle(
          fontSize: 12,
          color: const Color.fromARGB(255, 124, 63, 58),
        ), // TextStyle
      ), // Text
    ],
  ); // Row
}
```

```
// 左侧结构
Widget _buildLeft() {
  return Container(
    width: 100,
    height: 140,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(10),
      image: DecorationImage(
        image: AssetImage("lib/assets/home_cmd_inner.png"),
        fit: BoxFit.cover,
      ), // DecorationImage
    ), // BoxDecoration
  ); // Container
}
```

获取特惠推荐渲染(AI + 手动)

6.特惠推荐渲染

```
@override
Widget build(BuildContext context) {
  return Padding(
    padding: EdgeInsets.symmetric(horizontal: 10),
    child: Container(
      alignment: Alignment.center,
      padding: EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.blue,
        borderRadius: BorderRadius.circular(12),
        image: DecorationImage(
          image: AssetImage("lib/assets/home_cmd_sm.png"),
          fit: BoxFit.cover,
        ), // DecorationImage
      ), // BoxDecoration
      child: Column(
        children: [
          // 顶部内容
          _buildHeader(),
          SizedBox(height: 10),
          Row(
            children: [
              _buildLeft(),
```

```
Expanded(
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: _getChildrenList(),
  ), // Row
), // Expanded
```

```
List<Widget> _getChildrenList() {
  List<GoodsItem> list = _getDisplayItems(); // 取到前3条数据
  return List.generate(list.length, (int index) {
    return Column(
      children: [
        // ClipRRect 可以包裹子元素 裁剪图片设置圆角
        ClipRRect(
          borderRadius: BorderRadius.circular(8),
          child: Image.network(
            errorBuilder: (context, error, stackTrace) {
              // 返回一个新的部件替换原有图片
              return Image.asset(
                "lib/assets/home_cmd_inner.png",
                width: 100,
                height: 140,
                fit: BoxFit.cover,
              ); // Image.asset
            },
            list[index].picture,
            width: 100,
            height: 140,
            fit: BoxFit.cover,
          ), // Image.network
```

```
      list[index].picture,
      width: 100,
      height: 140,
      fit: BoxFit.cover,
    ), // Image.network
  ), // ClipRRect
  SizedBox(height: 10),
  Container(
    padding: EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(12),
      color: const Color.fromARGB(255, 240, 96, 12),
    ), // BoxDecoration
    child: Text(
      "¥${list[index].price}",
      style: TextStyle(color: Colors.white),
    ), // Text
  ), // Container
,
```

爆款推荐/一站买全（集成）

说明

爆款推荐和一站买全结构特惠推荐完全一致，使用同一份结构

步骤

1.请求地址常量

2.API请求

3.初始化获取数据

4.传递数据到子组件

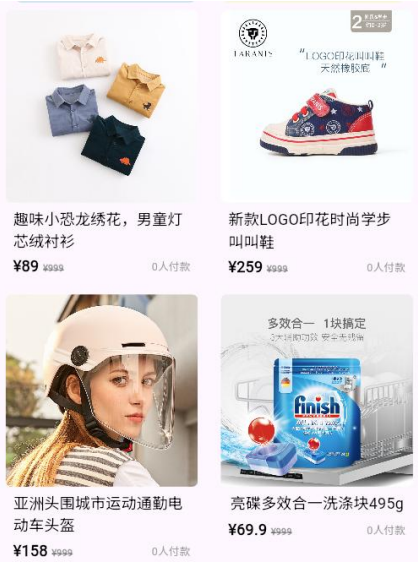
5.实现渲染视图

爆款推荐(get)：/hot/inVogue

一站买全(get)：/hot/oneStop

素材在课程资料中提供（同学也可以自己实现）

推荐列表(集成)



说明

1.集成素材获取第一页数据

2.limit：数量为查询商品数量

步骤

1.请求地址常量

2.API请求

3.初始化获取数据

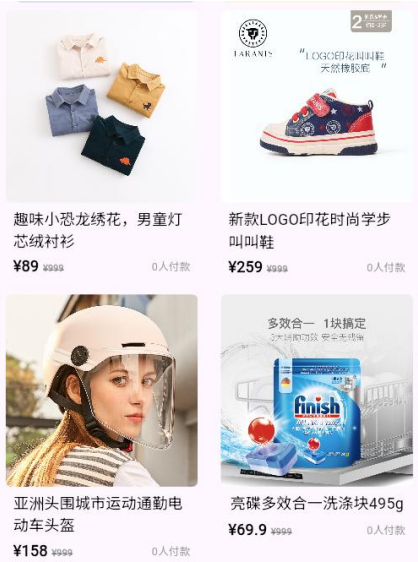
4.传递数据到子组件

5.实现渲染视图

推荐列表(get)：/hot/recommend

素材在课程资料中提供（同学也可以自己实现）

上拉加载



实现要素

1.使用原有接口实现

2.监听滚动到底部事件

3. 同时只能加载一个请求

4. 如果没有下一页不能再发起请求

1.定义控制器绑定组件

```
  // 声明滚动容器
  final ScrollController _scrollController = ScrollController();
  @override
  Widget build(BuildContext context) {
    return CustomScrollView(
      controller: _scrollController,
      slivers: _getScrollChildern(),
    ); // sliver家族的内容
  }
}
```

上拉加载

2.定义页码/加载状态/是否有下一页

```
int _recommendListPage = 1;
bool _hasMore = true;
bool _isLoading = false;
// 获取推荐列表
```

3.监听滚动到底部事件获取数据

```
// 监听滚动到底部的事件
void _registerEvent() {
  _controller.addListener(() {
    if (_controller.position.pixels >=
        (_controller.position.maxScrollExtent - 50)) {
      // print("到底啦");
      // 加载下一页数据
      _getRecommendList();
    }
  });
}
```

4.获取数据逻辑改造

```
void _getRecommendList() async {
  if (!_hasMore || _isLoading) {
    return;
  }
  _isLoading = true;
  int requestLimit = _recommendListPage * 10;
  _recommendList = await getRecommendListAPI({"limit": requestLimit});
  _isLoading = false;
  setState(() {});
  // 要的数据没给到
  if (_recommendList.length < requestLimit) {
    _hasMore = false;
    return;
  }

  _recommendListPage++;
}
```

下拉刷新/提示消息/GlobalKey

实现要素

1.使用RefreshIndicator包裹子组件，向下拉触发onRefresh函数

2. 数据重置，重新获取

3. 获取完毕提示消息（封装）

4. 初始化时调用下拉刷新动作

1.使用RefreshIndicator包裹自定义滚动容器

```
// 实现下拉刷新
return RefreshIndicator(
  onRefresh: _onRefresh,
  triggerMode: RefreshIndicatorTriggerMode.anywhere, // 触发模式
  child: CustomScrollView(
      controller: _controller, // 绑定控制器
      slivers: _getScrollChildern(),
    ), // sliver家族的内容 // CustomScrollView
); // RefreshIndicator
}
```

下拉刷新/提示消息/GlobalKey

2.实现下拉刷新方法

```dart
// 封装onRefresh () {}
Future<void> _onRefresh() async {
  // 下拉刷新时 重置页码 重新加载数据
  _page = 1;
  _isLoading = false;
  _hasMore = true;
  await _getProductList();
  await _getInVogueList();
  await _getOneStopList();
  await _getRecommendList();
  await _getBannderList();
  await _getCategoryList();
  print("刷新成功");
}
```

3.获取数据改造

```dart
// 获取热榜推荐列表      类型变成Future<void>
Future<void> _getInVogueList() async {
  _inVogueResult = await getInVogueListAPI();
}          去掉setState

// 获取一站式推荐列表
Future<void> _getOneStopList() async {
  _oneStopResult = await getOneStopListAPI();
}
```

下拉刷新/提示消息/GlobalKey

4.封装提示信息方法

```dart

class ToastUtils {
  static void showToast(BuildContext context, String msg) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        width: 120,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(40)),
        content: Text(msg, textAlign: TextAlign.center),
        behavior: SnackBarBehavior.floating, // 默认值
        duration: Duration(seconds: 3),
      ), // SnackBar
    );
  }
}
```

```dart
// 封装onRefresh () {}
Future<void> _onRefresh() async {
  // 下拉刷新时 重置页码 重新加载数据
  _page = 1;
  _isLoading = false;
  _hasMore = true;
  await _getProductList();
  await _getInVogueList();
  await _getOneStopList();
  await _getRecommendList();
  await _getBannderList();
  await _getCategoryList();
  setState(() {});
  ToastUtils.showToast(context, '刷新成功');
}
```

5.创建GlobalKey绑定下拉容器

```dart
final GlobalKey<RefreshIndicatorState> _refreshIndicatorKey =
    GlobalKey<RefreshIndicatorState>();
```

```dart
return RefreshIndicator(
  key: _refreshIndicatorKey,
  onRefresh: _onRefresh,
  triggerMode: RefreshIndicatorTriggerMode.anywhere, // 触发模式
```

```dart
@override
void initState() {
  // TODO: implement initState
  super.initState();

  _registerEvent();
  Future.microtask(() {
    _refreshIndicatorKey.currentState?.show();
  });
}
```

等到视图创建调用刷新

下拉刷新/提示消息/GlobalKey

6.动画过度

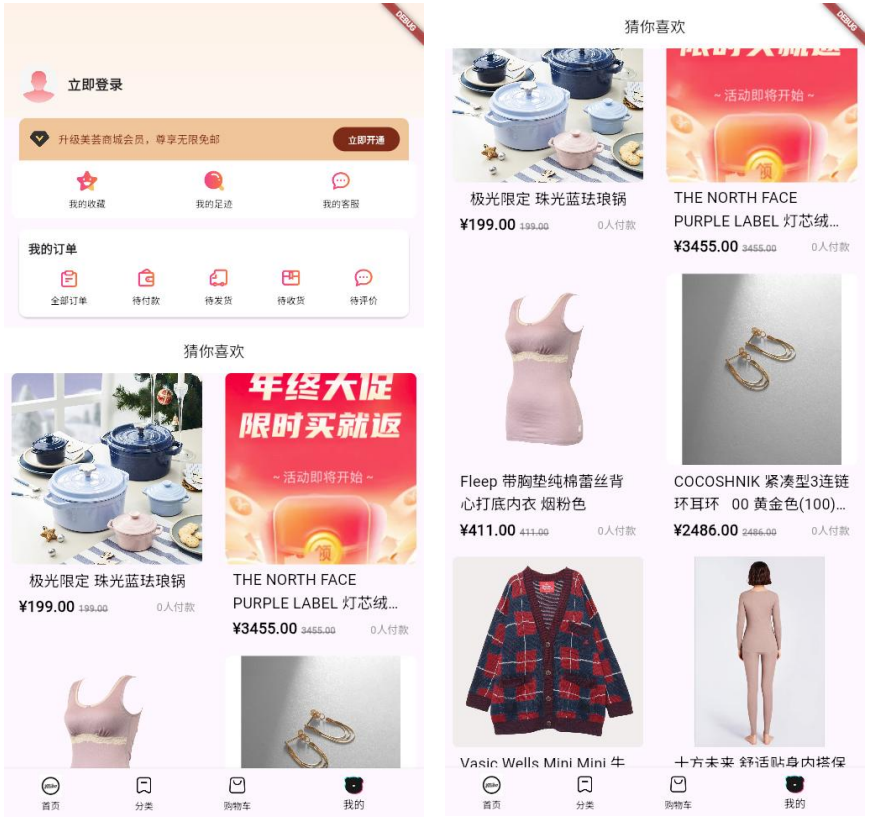
```dart
double _paddingTop = 0;
```

```dart
return RefreshIndicator(
  key: _refreshIndicatorKey,
  onRefresh: _onRefresh,
  triggerMode: RefreshIndicatorTriggerMode.anywhere, // 触发模式
  child: AnimatedContainer(
    duration: Duration(milliseconds: 300),
    padding: EdgeInsets.only(top: _paddingTop),
    child: CustomScrollView(
      controller: _controller, // 绑定控制器
      slivers: _getScrollChildern(),
    ), // CustomScrollView
  ), // sliver家族的内容 // AnimatedContainer
```

```dart
@override
void initState() {
  // TODO: implement initState
  super.initState();

  _registerEvent();
  Future.microtask(() {      You, 1秒钟前 • Uncommitted char
    _paddingTop = 50;
    setState(() {});
    _refreshIndicatorKey.currentState?.show();
  });
}
```

```dart
// 封装onRefresh () {}
Future<void> _onRefresh() async {
  // 下拉刷新时 重置页码 重新加载数据
  _page = 1;
  _isLoading = false;
  _hasMore = true;
  await _getProductList();
  await _getInVogueList();
  await _getOneStopList();
  await _getRecommendList();
  await _getBannderList();
  await _getCategoryList();
  _paddingTop = 0;
  setState(() {});
  // 刷新成功后 提示用户
  ToastUtils.showToast(context, '刷新成功');
}
```

我的页面(集成+搭建)



要素及功能

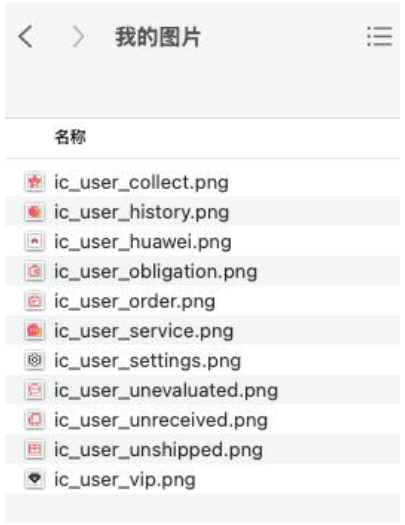
1.准备图片资源，使用集成素材快速搭建

2. 实现猜你喜欢滚动吸顶功能

3.实现猜你喜欢上拉加载功能

4. 复用原来的HmMoreList组件

1.拷贝我的素材到lib/assets目录下(图片拷贝之后需要重启)



我的页面(集成+搭建)

2.集成我的页面页面结构

3.构建猜你喜欢吸顶组件


```dart
@override
Widget build(BuildContext context) {
  return CustomScrollView(
    controller: _controller,
    slivers: [
      SliverToBoxAdapter(child: _buildHeader()),
      SliverToBoxAdapter(child: _buildVipCard()),
      SliverToBoxAdapter(child: _buildQuickActions()),
      SliverToBoxAdapter(child: _buildOrderModule()),
      // 猜你喜欢
      SliverPersistentHeader(pinned: true, delegate: HmGuess()),
      // HmMoreList(recommendList: _list),
    ],
  ); // CustomScrollView
}
}
```

我的页面(集成+搭建)

3.猜你喜欢代码结构

```dart
class HmGuess extends SliverPersistentHeaderDelegate {
  @override
  Widget build(BuildContext context,double shrinkOffset,bool overlapsContent) =>
    // TODO: implement build
     Container(
      color: Colors.white,
      alignment: Alignment.center,
      child: Text("猜你喜欢", style: TextStyle(fontSize: 20)),
    ); // Container


  @override
  // TODO: implement maxExtent
  double get maxExtent => 60;

  @override
  // TODO: implement minExtent
  double get minExtent => 60;

  @override
  bool shouldRebuild(covariant SliverPersistentHeaderDelegate oldDelegate) => false;
}
```

我的页面(集成+搭建)

4.定义猜你喜欢常量地址

```
You, 3小时前 | 1 author (You)
class HttpConstants {
  static const String BANNER_LIST = "/home/banner";
  static const String CATEGORY_LIST = "/home/category/head"; // 分类列表
  static const String PRODUCT_LIST = "/hot/preference"; // 特惠推荐地址
  static const String IN_VOGUE_LIST = "/hot/inVogue"; // 热榜推荐地址
  static const String ONE_STOP_LIST = "/hot/oneStop"; // 一站式推荐地址
  static const String RECOMMEND_LIST = "/home/recommend"; // 推荐列表
  static const String GUESS_LIST = "/home/goods/guessLike"; // 猜你喜欢
}
```

注意：猜你喜欢同特惠推荐的GoodsItems结构完全一样

但是HmMoreList的数据用的是GoodDetailItem,而GoodsItems中的数据是GoodItem，所以需要转化

5.定义猜你喜欢数据结构（基于GoodsItems修改）

```
class GoodsDetailItems {
  int counts;
  int pageSize;
  int pages;
  int page;
  List<GoodDetailItem> items;
  GoodsDetailItems({      You, 2小时前 • Uncommitted changes
    required this.counts,
    required this.pageSize,
    required this.pages,
    required this.page,
    required this.items,
  });
  factory GoodsDetailItems.formJSON(Map<String, dynamic> json) {
    return GoodsDetailItems(
      counts: int.tryParse(json["counts"]?.toString() ?? "0") ?? 0,
      pageSize: int.tryParse(json["pageSize"]?.toString() ?? "0") ?? 0,
      pages: int.tryParse(json["pages"]?.toString() ?? "0") ?? 0,
      page: int.tryParse(json["page"]?.toString() ?? "0") ?? 0,
      items: (json["items"] as List? ?? [])
          .map((item) => GoodDetailItem.formJSON(item as Map<String, dynamic>))
          .toList(),
    ); // GoodsDetailItems
  }
}
```

我的页面(集成+搭建)

5.封装API

```
Future<GoodsDetailItems> getGuessListAPI(Map<String, dynamic> params) async {
  return GoodsDetailItems.formJSON(
    await dioRequest.get(HttpConstants.GUESS_LIST, params: params),
  );
}
```

6.定义基础数据

```
List<GoodDetailItem> _list = [];      You, 1秒钟前 • Uncommitted chan
bool _isLoading = false;
bool _harMore = true;
Map<String, dynamic> _params = {"page": 1, "pageSize": 10};
```

我的页面(集成+搭建)

7.实现加载方法

```
void _getGuessList() async {
  if (!_harMore || _isLoading) {
    return;
  }
  _isLoading = true;
  GoodsDetailItems result = await getGuessListAPI(_params);
  _list.addAll(result.items);
  _isLoading = false;
  setState(() {});
  if (result.pages <= _params["page"]) {
    _harMore = false;
    return;
  }
  _params["page"]++;
}
```

8.实现监听滚动加载

```
final ScrollController _controller = ScrollController();
```

```
@override
Widget build(BuildContext context) {
  return CustomScrollView(
    controller: _controller,
    slivers: [
```

```
void _registerEvent() {
  _controller.addListener(() {
    if (_controller.position.pixels <=
        (_controller.position.maxScrollExtent - 50)) {
      _getGuessList();
    }
  });
}
```

登录页面(集成+表单校验)

要素及功能

1.使用提供的登录页面结构（建议同学自己实现）

2. 我的页面 => 登录页

3.实现账号/密码/勾选的校验并提示

4. 实现按钮的校验控制

5. 优化轻提示工具

知识点

1.Form组件 => TextFormField 可实现表单校验

2. 使用GlobalKey创建key绑定Form组件可调用valiate方法

3. TextFormField的validator返回校验文本

4. 正则表达式使用RegExp(r'')进行校验

登录页面(集成+表单校验)

1.集成登录页素材

2.我的页面跳转到登录


```
children: [
  GestureDetector(
    onTap: () {
      Navigator.pushNamed(context, "/login");
    },
    child: Text(
      '立即登录',
      style: TextStyle(fontSize: 18, fontWeight: FontWeight.
      w600),
    ), // Text      You, 2小时前 • Uncommitted changes
  ), // GestureDetector
```

登录页面(集成+表单校验)

2.手机号和验证码校验逻辑

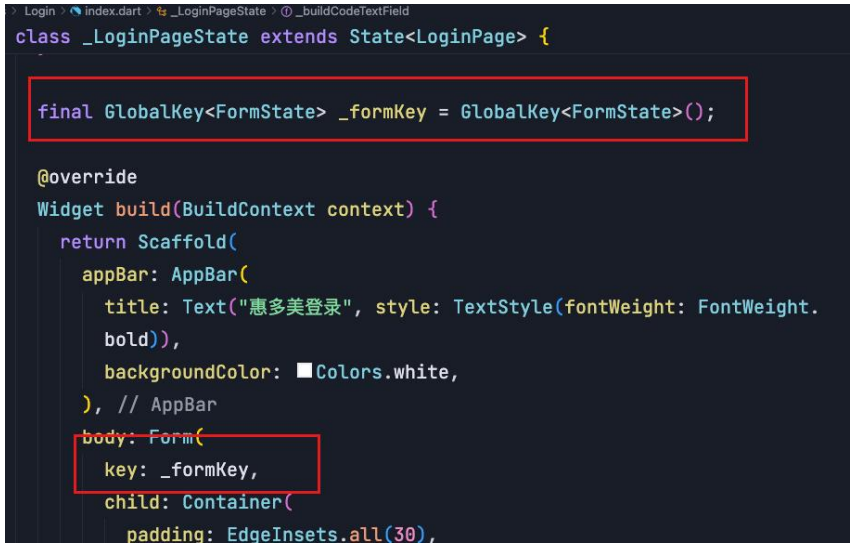

```
return TextFormField(
  controller: _phoneController,
  validator: (value) {
    if (value == null || value.isEmpty) {
      return "请输入账号";
    }
    // 校验手机号格式
    // 换成Pattern
    if (!RegExp(r'^1[3-9]\d{9}$').hasMatch(value)) {
      return "请输入正确的手机号";
    }
    return null;
  },
```

```
return TextFormField(
  controller: _codeController,
  obscureText: true,
  validator: (value) {
    if (value == null || value.isEmpty) {
      return "请输入密码";
    }
    // 校验密码格式
    if (!RegExp(r'^[a-zA-Z0-9_]{6,12}$').hasMatch(value)) {
      return "请输入6-12位字母、数字或下划线";
    }
    return null;
  },
```

```
Checkbox(
  value: _isChecked,
  activeColor: Colors.black,
  checkColor: Colors.white,
  onChanged: (bool? value) {
    setState(() {
      _isChecked = value ?? false;
    });
  },
  // 设置形状
```

登录页面(集成+表单校验)

3.点击按钮校验



```
Login > index.dart > _LoginPageState > _buildCodeTextField
class _LoginPageState extends State<LoginPage> {

  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("惠多美登录", style: TextStyle(fontWeight: FontWeight.
        bold)),
        backgroundColor: Colors.white,
      ), // AppBar
      body: Form(
        key: _formKey,
        child: Container(
          padding: EdgeInsets.all(30),
```

```
child: ElevatedButton(
  onPressed: () {
    // 登录逻辑
    if (_formKey.currentState!.validate()) {
      // 校验通过，执行登录操作
      print("账号: ${_phoneController.text}");
      print("密码: ${_codeController.text}");

      if (_isChecked) {
        // 勾选了同意协议，执行登录操作
      } else {
        // 未勾选同意协议，提示用户勾选
        ToastUtils.showToast(context, "请先勾选同意协议");
      }
    }
  },
```

登录页面(集成+表单校验)

4.改造轻提示

```dart
class ToastUtils {
  // 保证当前只有一个tiToast显示
  static bool _isShowing = false;
  static void showToast(BuildContext context, String? msg) {
    if (_isShowing) {
      return;
    }      You, 34分钟前 • Uncommitted changes
    _isShowing = true;
    // 延时3秒后隐藏Toast
    Future.delayed(Duration(seconds: 3), () {
      _isShowing = false;
    }); // Future.delayed
```

登录实现

登录接口(post)：/login

账号
account:13200000001~13200000010

密码
password:123456

要素及功能

1.登录接口地址

2. 定义登录接口返回类型(素材提供)

3.请求工具实现post方法

4. 封装登录API

5. 登录并捕获登录异常提示

1.定义常量地址

```
You, 2小时前 | 1 author (You)
class HttpConstants {
  static const String BANNER_LIST = "/home/banner";
  static const String CATEGORY_LIST = "/home/category/head"; // 分类列表
  static const String PRODUCT_LIST = "/hot/preference"; // 特惠推荐地址
  static const String IN_VOGUE_LIST = "/hot/inVogue"; // 热榜推荐地址
  static const String ONE_STOP_LIST = "/hot/oneStop"; // 一站式推荐地址
  static const String RECOMMEND_LIST = "/home/recommend"; // 推荐列表
  static const String GUESS_LIST = "/home/goods/guessLike"; // 猜你喜欢接口
  地址
  static const String LOGIN = "/login"; // 登录接口地址
  // 返回的结构体 是GoodsItems类型
}
```

登录实现

2.集成登录接口返回类型

```dart
// 生成对应的class
class UserInfo {
  String account;
  String avatar;
  String birthday;
  String cityCode;
  String gender;
  String id;
  String mobile;
  String nickname;
  String profession;
  String provinceCode;
  String token;

  UserInfo({
    required this.account,
    required this.avatar,
    required this.birthday,
    required this.cityCode,
    required this.gender,
    required this.id,
    required this.mobile,
    required this.nickname,
    required this.profession,
    required this.provinceCode,
    required this.token
```

3.dioRequest提供post方法

```dart
// post请求
Future<dynamic> post(String url, {Map<String, dynamic>? data}) {
  return _handleResponse(_dio.post(url, data: data));
}
```

4.封装登录API

```dart
Future<UserInfo> loginAPI(Map<String, dynamic> params) async {
  return UserInfo.fromJSON(
    await dioRequest.post(HttpConstants.LOGIN, data: params),
  );
}
```

登录实现

5.实现登录方法-捕获异常

```dart
if (_isChecked) {
  // 校验通过 调用登录方法
  _login();
} else {
  // 提示请勾选用户协议
  ToastUtils.showToast(context, "请勾选用户协议");
}
}
```

```dart
_login() async {
  // 调用登录接口
  try {
    final res = await loginAPI({
      "account": _phoneController.text,
      "password": _codeController.text,
    });
    print(res); // 用户信息
    ToastUtils.showToast(context, "登录成功");
    Navigator.pop(context); // 返回上个页面
  } catch (e) {
    // 弹出登录失败的消息 DioException
    // Exception转化成DioException
    ToastUtils.showToast(context, (e as DioException).message);
  }

  // 此时一定登录成功
```

登录实现

6.改造DioRequest

```
    },
    onError: (error, handler) {
      // handler.reject(error);
      handler.reject(
        DioException(
          requestOptions: error.requestOptions,
          message: error.response?.data["msg"] ?? "异常信息",
        ), // DioException
      );
    },
  ), // InterceptorsWrapper       You, 上周 • 获取轮播图数据
);
```

```
// 进一步处理返回结果的函数
Future<dynamic> _handleResponse(Future<Response<dynamic>> task) async {
  try {
    Response<dynamic> res = await task;
    final data = res.data as Map<String, dynamic>; // data才是我们真实的接口返回的数据
    if (data["code"] == GlobalConstants.SUCCESS_CODE) {
      // 才认定 http状态和业务状态均正常 就可以正常的放行通过
      return data["result"]; // 只要result结果
    }
    // 抛出异常
    // throw Exception(data["msg"] ?? "加载数据异常");
    throw DioException(
      requestOptions: res.requestOptions,
      message: data["msg"] ?? "加载数据异常",
    );
  } catch (e) {
    // 保持原始异常类型（如拦截器抛出的 DioException），不再包裹为通用 Exception
    rethrow;
  }
}
```

Getx共享用户数据

Getx用法

1.安装getx插件

2. 定义一个继承GetxController的对象

3.对象中定义需要共享的属性

4. 数据需要响应式更新-需要给初始值.obs结尾

5. UI显示Getx数据需使用Obx包裹函数中使用

6.UI中使用Getx需要一个put和find动作

7.必须先put一次，才可以find

8.put仅一次，find可多次

```
安装getx: flutter pub add get
```

Getx共享用户数据

1.定义UserController对象

```
class UserController extends GetxController {
  var user = UserInfo.fromJSON({}).obs;
  void updateUserInfo(UserInfo newUser) {
    user.value = newUser;
  }
}
```

.obs表示该数据为响应式数据

被.obs修饰的数据包了一层，赋值直接赋值value

2.在MineView中put控制器

```
class _MineViewState extends State<MineView> {
  final UserController _userController = Get.put(UserController());
```

```
Obx(() {
  return CircleAvatar(
    radius: 26,
    backgroundImage: _getAvatar(_userController.user.value.avatar),
    backgroundColor: Colors.white,
  ); // CircleAvatar
}), // Obx
```

```
Obx(() {
  return GestureDetector(
    onTap: () {
      if (_userController.user.value.account.isEmpty) {
        Navigator.pushNamed(context, "/login");
      }
    },
    child: Text(
      _userController.user.value.account.isNotEmpty
          ? _userController.user.value.account
          : "立即登录",
      // '立即登录',
      style: TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.w600,
      ), // TextStyle
    ), // Text
  ); // GestureDetector
}), // Obx
```

Getx共享用户数据

3.登录成功find控制器并更新数据

```
UserController _userController = Get.find<UserController>();
_login() async {
  // 调用登录接口
  try {
    final res = await loginAPI({
      "account": _phoneController.text,
      "password": _codeController.text,
    });      You, 4小时前 • 登录功能并提示异常信息 …
    print(res); // 用户信息
    _userController.updateUserInfo(res);
    ToastUtils.showToast(context, "登录成功");
    Navigator.pop(context); // 返回上个页面
  } catch (e) {
    ToastUtils.showToast(context, (e as DioException).message);
  }
  // 此时一定登录成功
  // http状态吗 2xx 业务状态吗 业务执行成功 1
}
```

Token持久化

持久化Token

1.安装shared_preferences插件

2. 封装一个TokenManager工具，具备初始化/设置/获取/删除方法

3.登录成功将token写入持久化

4.封装获取用户信息API

5.Dio在请求工具中进行token注入

6. 在应用首页判断token获取状态赋值Getx数据

7. 调整我的页面的Getx的put方式为find方式

```
安装持久化插件: flutter pub add shared_preferences
```

```
用户信息接口(get): /member/profile
```

Token持久化

1.安装插件

安装持久化插件：flutter pub add shared_preferences

2.封装TokenManager对象

```dart
class TokenManager {
  String _token = "";  管理token
  Future<SharedPreferences> _getInstance() {
    return SharedPreferences.getInstance();     获取持久化实例对象
  }
 Future<void> init() async {  初始化token
    final prefs = await _getInstance();
    _token = prefs.getString(GlobalConstants.TOKEN_KEY) ?? "";
  }
  Future<void> setToken(String val) async {  设置token
    final prefs = await _getInstance();
    _token = val;
    prefs.setString(GlobalConstants.TOKEN_KEY, val);
  }
  String getToken() {
    return _token;     获取token
  }
  Future<void> removeToken() async {       删除token
    final prefs = await _getInstance();
    _token = "";
    prefs.remove(GlobalConstants.TOKEN_KEY);
  }
}
final tokenManager = TokenManager();
```

3.登录成功写入token

```dart
_login() async {
  // 调用登录接口
  try {
    final res = await loginAPI({
      "account": _phoneController.text,
      "password": _codeController.text,
    });
    // print(res); // 用户信息
    _userController.updateUserInfo(res);
    tokenManager.setToken(res.token);
    ToastUtils.showToast(context, "登录成功");
    Navigator.pop(context); // 返回上个页面
  } catch (e) {
    ToastUtils.showToast(context, (e as DioException).message);
  }
  // 此时一定登录成功
  // http状态吗 2xx 业务状态吗 业务执行成功 1
}
```

Token持久化

4.封装获取用户信息API

```
static const String USER_PROFILE = "/member/profile"; // 用户信息接口
```

```
Future<UserInfo> getUserInfoAPI() async {
  return UserInfo.fromJSON(await dioRequest.get(HttpConstants.USER_PROFILE));
}
```

5.Dio拦截器注入token

```
onRequest: (request, handler) {
  if (tokenManager.getToken().isNotEmpty) {
    request.headers = {
      "Authorization": 'Bearer ${tokenManager.getToken()}',
    };
  }
  handler.next(request);
},
```

Token持久化

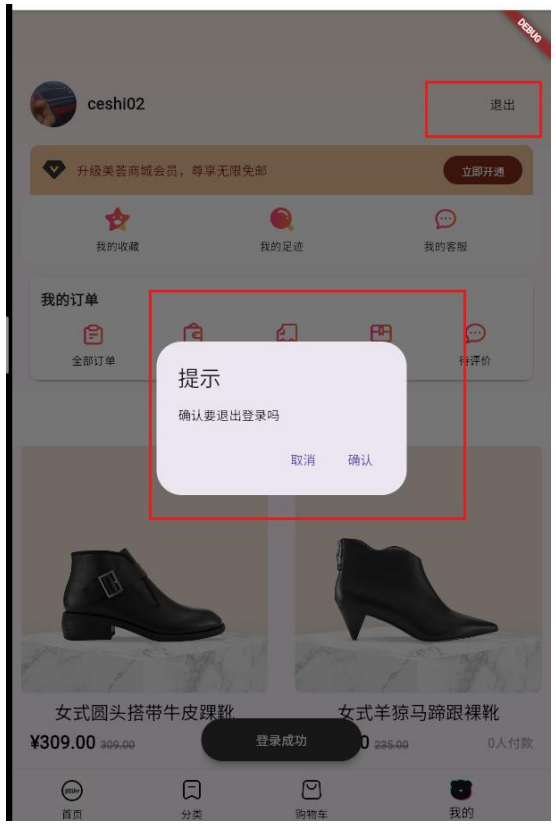
6.主页初始化获取用户信息

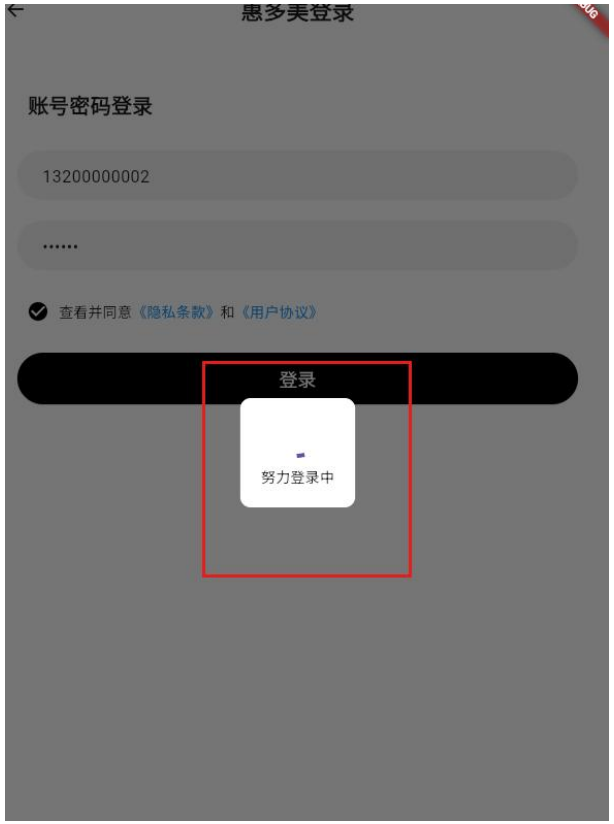
```dart
final UserController _userController = Get.put(UserController());
@override
void initState() {
  // TODO: implement initState
  super.initState();
  initUser();
}

initUser() async {
  await tokenManager.init();
  if (tokenManager.getToken().isNotEmpty) {
    _userController.user.value = await getUserInfoAPI();
  }
}
```

7.我的页面Getx调整层find

```dart
final UserController _userController = Get.find();
```

退出登录–确认弹窗–进度弹窗





退出登录

1.提示确认弹窗

2. 点击取消关闭弹窗

3.点击确认-清空Getx数据-清除token-关闭弹窗

弹窗用法

1.showDialog -> 传入builder(Dialog基础类扩展)

2. 使用路由的Navigator.pop关闭即可

退出登录–确认弹窗–进度弹窗

1.退出登录逻辑

```
child: Row(
  children: [
    Obx(() { // Obx ⋯
    const SizedBox(width: 12),
    Expanded( // Expanded ⋯
    Obx(() => _getLogOut()),
  ],
), // Row
```

```
Widget _getLogOut() {
  return _userController.user.value.id.isNotEmpty
      ? Expanded(
          child: GestureDetector(
            onTap: () {
              showDialog(
                context: context,
                builder: (BuildContext context) {
                  return AlertDialog(
                    title: Text("提示"),
                    content: Text("确认要退出登录吗"),
                    actions: [
                      TextButton(
                        onPressed: () {
                          Navigator.pop(context);
                        },
                        child: Text("取消"),
                      ), // TextButton
                      TextButton(
                        onPressed: () async {
                          await tokenManager.removeToken();
                          _userController.updateUserInfo(
                            UserInfo.fromJSON({}),
                          );
                          Navigator.pop(context);
                        },
```

退出登录–确认弹窗–进度弹窗

2.封装Loading弹层

```dart
class LoadingDialog {
  static void show(BuildContext context, {String message = '加载中...'}) {
    showDialog(
      context: context,
      barrierDismissible: false, // 点击背景不关闭
      builder: (context) => Dialog(
        backgroundColor: Colors.transparent,
        elevation: 0,
        child: Center(
          child: Container(
            padding: EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(10),
            ), // BoxDecoration
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                CircularProgressIndicator(),
                SizedBox(height: 10),
                Text(message),
              ],
            ), // Column
          ), // Container
        ), // Center
      ), // Dialog
```

```dart
_login() async {
  // 调用登录接口
  try {
    LoadingDialog.show(context, message: "努力登录中");
    final res = await loginAPI({
      "account": _phoneController.text,
      "password": _codeController.text,
    });
    _userController.updateUserInfo(res);      You, 昨天 • 使用Getx完成数据的
    tokenManager.setToken(res.token); // 写入持久化数据

    ToastUtils.showToast(context, "登录成功");
    LoadingDialog.hide(context);
    Navigator.pop(context); // 返回上个页面
  } catch (e) {
    ToastUtils.showToast(context, (e as DioException).message);
  }
  // 此时一定登录成功
  // http状态吗 2xx 业务状态吗 业务执行成功 1
}
```

安卓平台适配

安卓环境搭建

适配步骤

1. 安装jdk17版本
2. 安装Android Studio
3. 设置Android sdk环境变量
4. 安装Dart和Flutter插件
5. 安装模拟器
6. 检测环境

检测环境命令：flutter doctor -v

```
[✗] Android toolchain - develop for Android devices [421ms]
    ✗ Unable to locate Android SDK.
      Install Android Studio from:
      https://developer.android.com/studio/index.html
      On first launch it will assist you in installing the Android SDK
      components.
      (or visit https://flutter.dev/to/macos-android-setup for detailed
      instructions).
      If the Android SDK has been installed to a custom location, please use
      `flutter config --android-sdk` to update to that location.
```

未配置环境之前

安卓适配–1.安装jdk17

Oracle官网下载连接：https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html

Linux x64 RPM Package	174.03 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_linux-x64_bin.rpm (sha256)
macOS Arm 64 Compressed Archive	168.84 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_macos-aarch64_bin.tar.gz (sha256)
macOS Arm 64 DMG Installer	168.24 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_macos-aarch64_bin.dmg (sha256)
macOS x64 Compressed Archive	170.91 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_macos-x64_bin.tar.gz (sha256)
macOS x64 DMG Installer	170.32 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_macos-x64_bin.dmg (sha256)
Windows x64 Compressed Archive	172.87 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_windows-x64_bin.zip (sha256)
Windows x64 Installer	153.92 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_windows-x64_bin.exe (sha256)
Windows x64 MSI Installer	152.67 MB	https://download.oracle.com/java/17/archive/jdk-17.0.12_windows-x64_bin.msi (sha256)

1.双击.exe或者.dmg文件，点击下一步，完成安装

2.配置环境变量为jdk下bin路径

查看jdk安装成功 javac --version

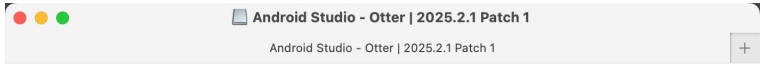
```
gaolingyu@MacBook-Pro-4 ~ % javac --version
javac 17.0.15
gaolingyu@MacBook-Pro-4 ~ %
```

安卓适配–2.安装AndroidStudio

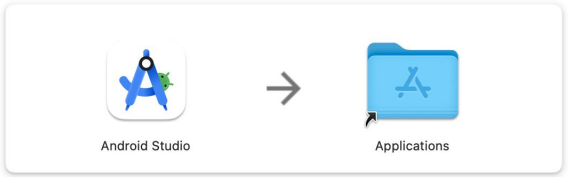
下载连接：https://developer.android.google.cn/studio?hl=zh-cn

注意　：1.请确保你的电脑的内存大于 8个G

2.软件安装路径，SDK路径，项目路径，都不要包含中文、空格、特殊字符



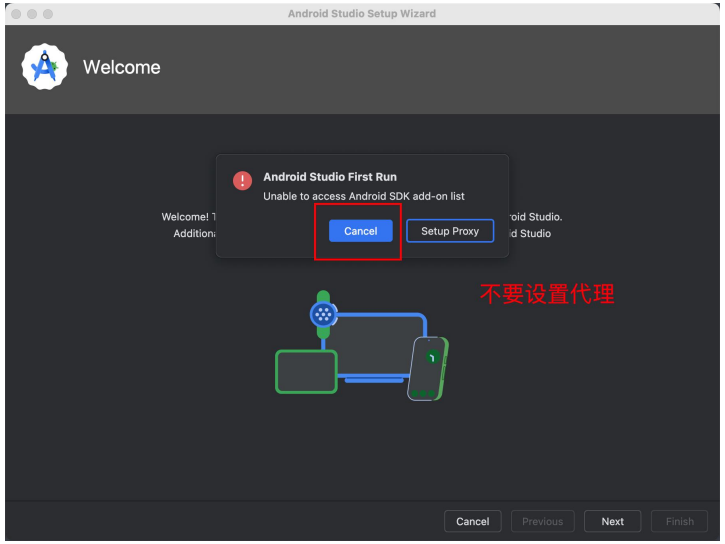
android studio

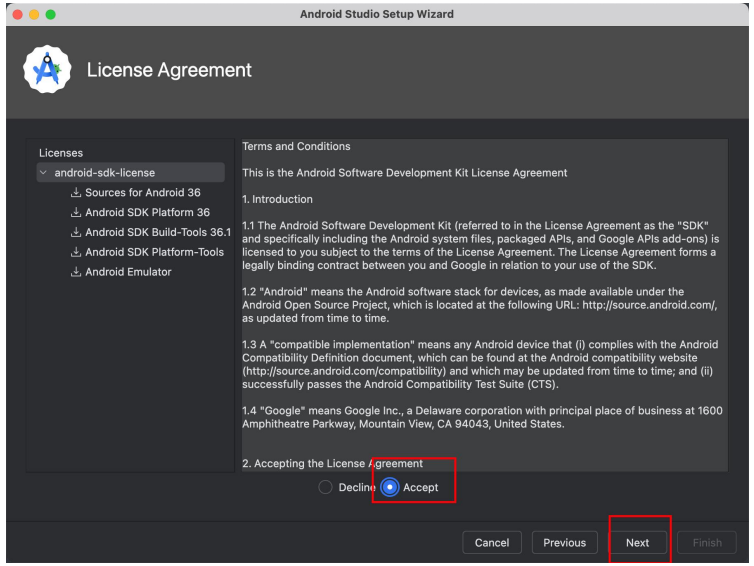


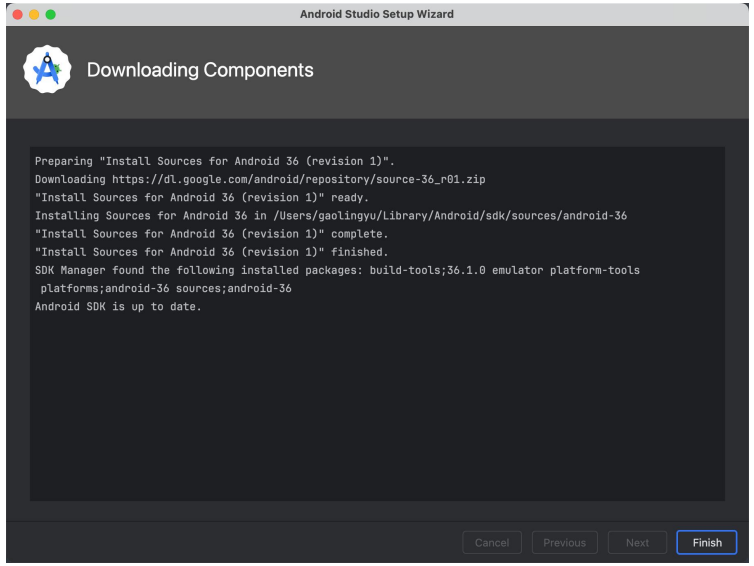
mac-> 拖入到Applications

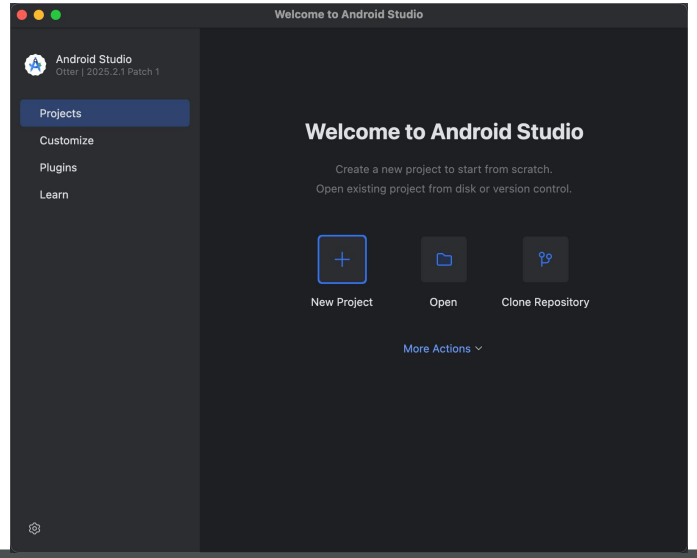
windows -> 点击next -> 安装

安卓适配–2.安装AndroidStudio

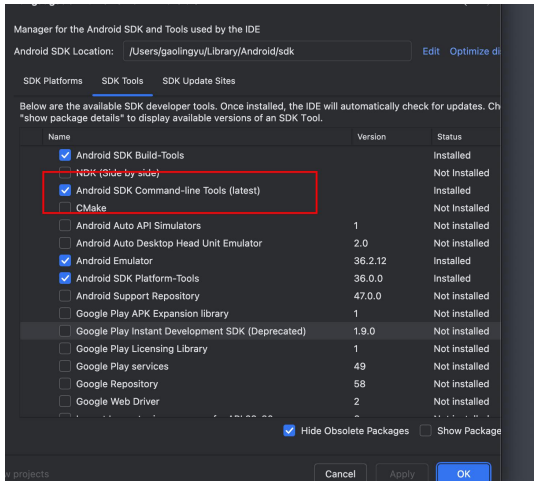


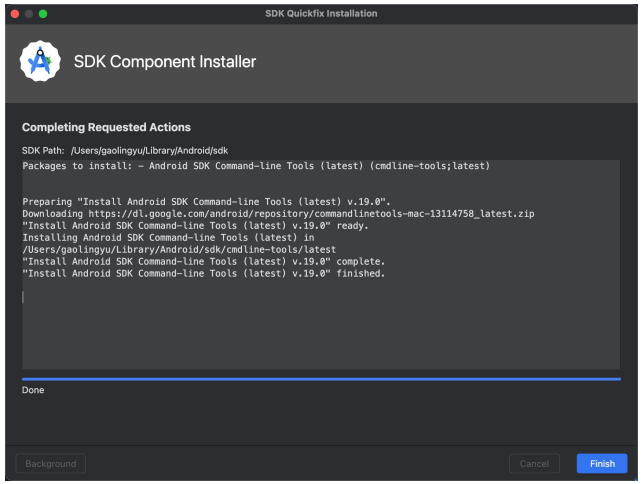






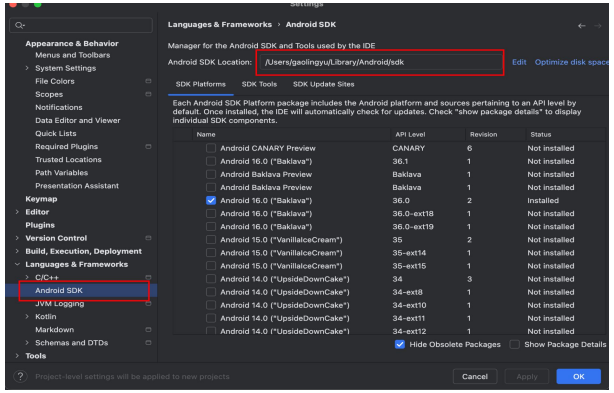
安卓适配-3.设置安卓SDK环境变量





接受安卓许可协议(一直选y即可)

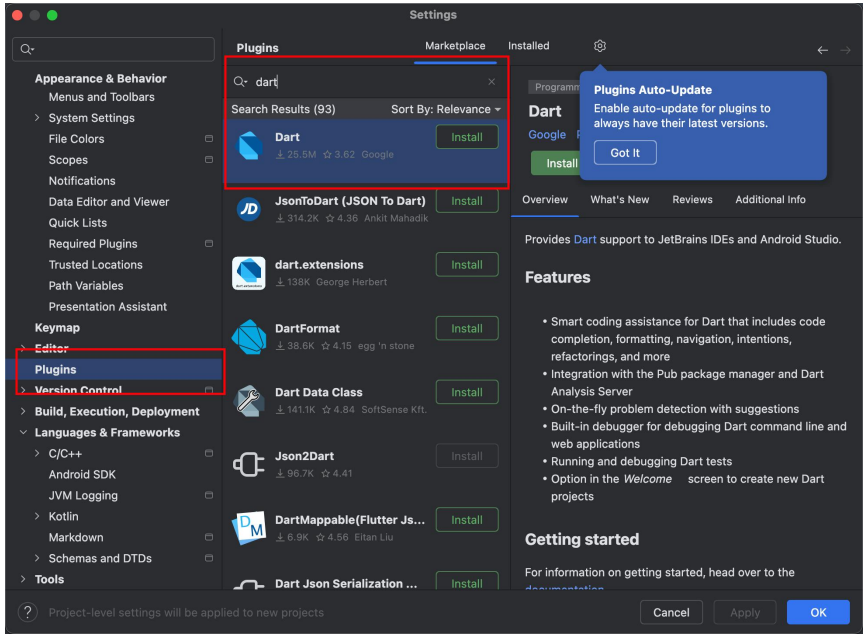
```
flutter doctor --android-licenses
```

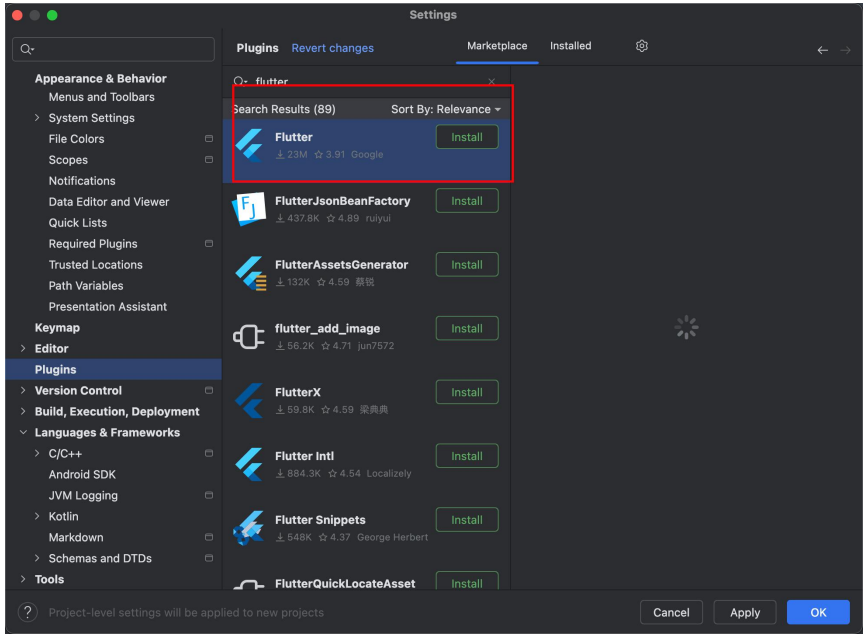



```
export ANDROID_HOME=/Users/xxx/Library/Android/sdk
```

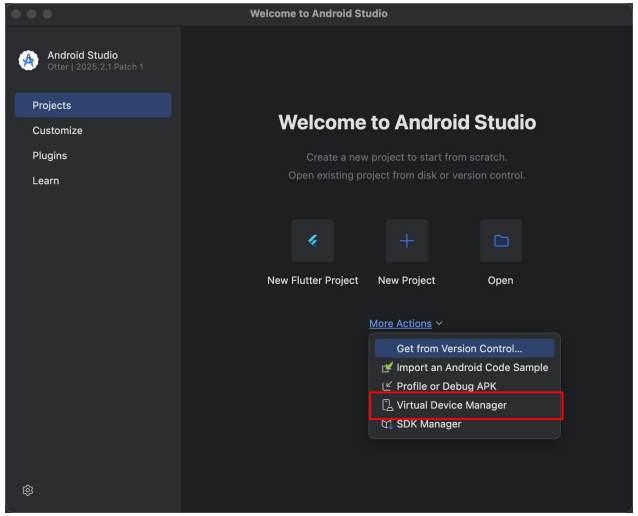
```
source ~/.zshrc
```

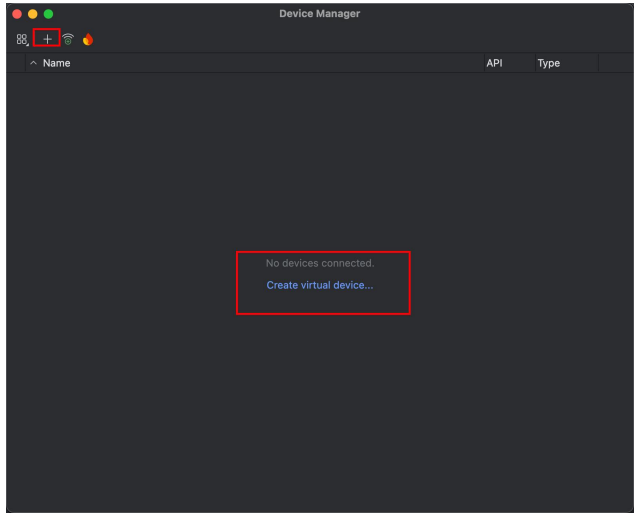
安卓适配–4.安装dart和flutter插件

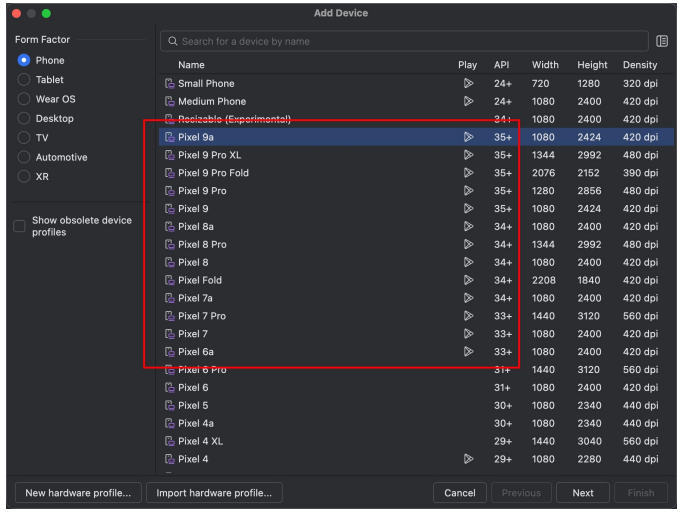


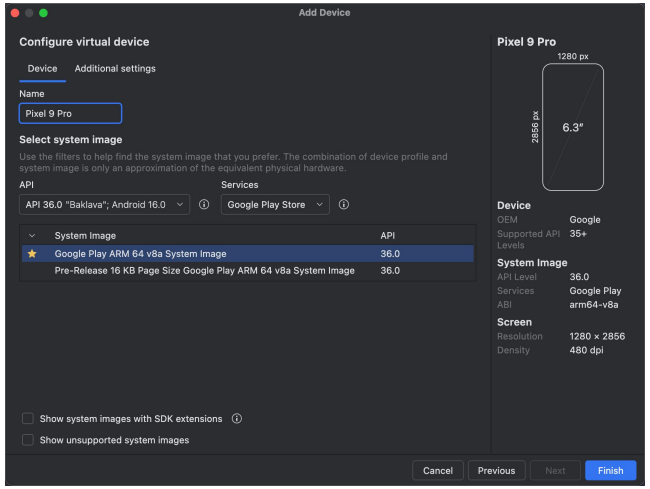


安卓适配-5.安装模拟器


Welcome to Android Studio
Android Studio
Otter | 2025.2.1 Patch 1
Projects
Customize
Plugins
Learn
Welcome to Android Studio
Create a new project to start from scratch.
Open existing project from disk or version control.
New Flutter Project
New Project
Open
More Actions
Get from Version Control...
Import an Android Code Sample
Profile or Debug APK
Virtual Device Manager
SDK Manager


Device Manager
Name
API
Type
No devices connected.
Create virtual device...


Add Device
Form Factor
Phone
Tablet
Wear OS
Desktop
TV
Automotive
XR
Show obsolete device profiles
Search for a device by name
Name
Play
API
Width
Height
Density
Small Phone
Medium Phone
Pixel 9a
Pixel 9 Pro XL
Pixel 9 Pro Fold
Pixel 9 Pro
Pixel 9
Pixel 8a
Pixel 8 Pro
Pixel 8
Pixel Fold
Pixel 7a
Pixel 7 Pro
Pixel 7
Pixel 6a
Pixel 6
Pixel 5
Pixel 4a
Pixel 4 XL
Pixel 4
New hardware profile...
Import hardware profile...
Cancel
Previous
Next
Finish


Add Device
Configure virtual device
Device
Additional settings
Name
Pixel 9 Pro
Select system image
Use the filters to help find the system image that you prefer. The combination of device profile and system image is only an approximation of the equivalent physical hardware.
API
Services
API 36.0 "Baklava"; Android 16.0
Google Play Store
System Image
API
Google Play ARM 64 v8a System Image
36.0
Pre-Release 16 KB Page Size Google Play ARM 64 v8a System Image
36.0
Show system images with SDK extensions
Show unsupported system images
Pixel 9 Pro
1280 px
2856 px
6.3"
Device
OEM
Google
Supported API Levels
35+
System Image
API Level
36.0
Services
Google Play
ABI
arm64-v8a
Screen
Resolution
1280 x 2856
Density
480 dpi
Cancel
Previous
Next
Finish


Tue, Nov 25
Play Store
Gmail
Photos
YouTube

安卓适配-6.检测环境

检测环境命令：flutter doctor -v

```
[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
    [3.3s]
    • Android SDK at /Users/gaolingyu/Library/Android/sdk
    • Emulator version 36.2.12.0 (build_id 14214601) (CL:N/A)
    • Platform android-36, build-tools 36.1.0
    • ANDROID_HOME = /Users/gaolingyu/Library/Android/sdk
    • Java binary at: /Applications/Android
      Studio.app/Contents/jbr/Contents/Home/bin/java
      This is the JDK bundled with the latest Android Studio installation on
      this machine.
      To manually set the JDK path, use: `flutter config
      --jdk-dir="path/to/jdk"`.
    • Java version OpenJDK Runtime Environment (build 21.0.8+-14196175-b1038.72)
    • All Android licenses accepted.
```

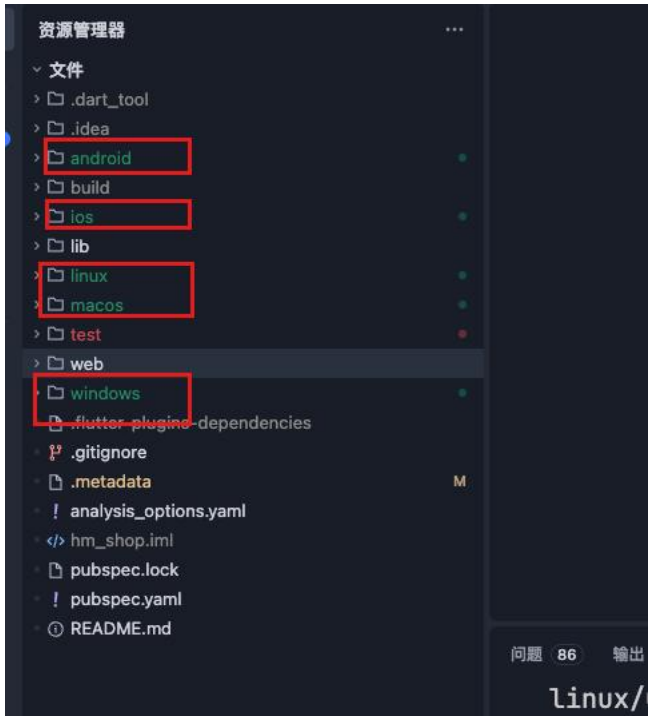
运行flutter项目到安卓端

运行步骤

1. 补全其它平台代码
2. 设置安卓gradle国内镜像和maven仓库
3. 设置安卓网络权限
4. 运行到模拟器
5. 解决UI超出问题
6. 打包

运行flutter项目到安卓端–1.补全其它平台代码

```
flutter create .
```



注意　：这里并没有ohos鸿蒙平台，因为官方flutter版本并不支持鸿蒙平台，需要特定的flutter版本才可以

运行flutter项目到安卓端–2.设置gradle和maven仓库(一定要做！)

1.打开android/gradle/wrapper/gradle-wrapper.properties设置distributionUrl为国内镜像地址

```
distributionUrl=https\://mirrors.cloud.tencent.com/gradle/gradle-8.14-all.zip
```

2.修改gradle的maven仓库镜像地址(android/settings.gradle.kts)

```
maven { url = uri("https://maven.aliyun.com/repository/google") }
maven { url = uri("https://maven.aliyun.com/repository/releases") }
maven { url = uri("https://maven.aliyun.com/repository/central") }
maven { url = uri("https://maven.aliyun.com/repository/public") }
maven { url = uri("https://maven.aliyun.com/repository/gradle-plugin") }
maven { url = uri("https://maven.aliyun.com/repository/apache-snapshots") }
maven { url = uri("https://maven.aliyun.com/nexus/content/groups/public/") }
maven { url = uri("https://jitpack.io") }
```

运行flutter项目到安卓端–3.设置网络权限

1.给安卓侧添加网络权限（android/app/src/main/AndroidManifest.xml）

```
<uses-permission android:name="android.permission.INTERNET" />
```

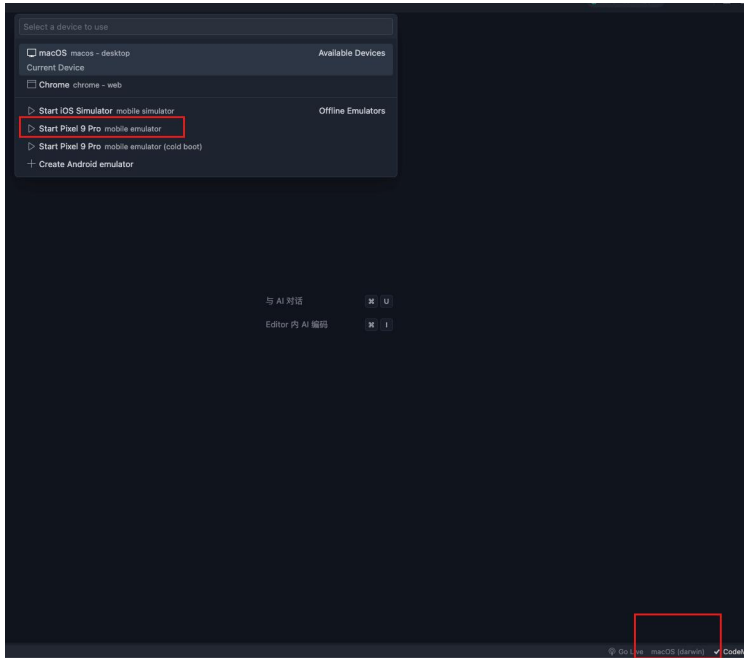
AndroidManifest.xml

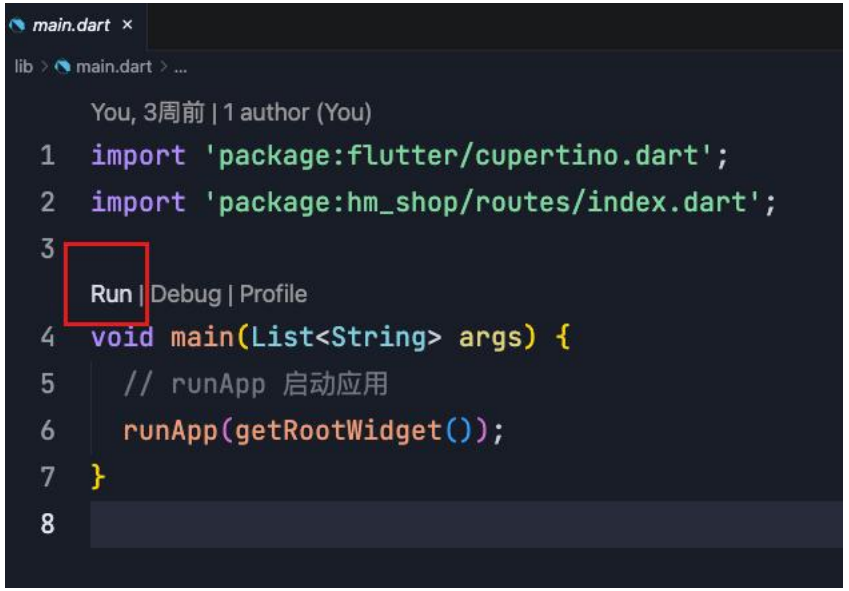
android > app > src > main > AndroidManifest.xml

```
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2      <uses-permission android:name="android.permission.INTERNET" />
3      <application
4          android:label="hm_shop"
5          android:name="${applicationName}"
6          android:icon="@mipmap/ic_launcher">
7          <activity
8              android:name=".MainActivity"
9              android:exported="true"
10             android:launchMode="singleTop"
11             android:taskAffinity=""
12             android:theme="@style/LaunchTheme"
13             android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smal
               layoutDirection|fontScale|screenLayout|density|uiMode"
14             android:hardwareAccelerated="true"
15             android:windowSoftInputMode="adjustResize">
```

运行flutter项目到安卓端–4.运行到模拟器

1.trae编辑器运行





运行flutter项目到安卓端-5.解决UI溢出

1.HmSuggestion


```
Row(
  children: [
    _buildLeft(),
    SizedBox(width: 10),
    Expanded(
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        spacing: 10,
        children: _getChildrenList(),
      ), // Row
    ), // Expanded        You, 2周前 • 实现特惠推荐的渲染
  ],
) // Row
```

```
return Expanded(
  child: Column(
    children: [
      // ClipRRect 可以包裹子元素 裁剪图片设置圆角
      ClipRRect(
        borderRadius: BorderRadius.circular(8),
        child: Image.network(
          errorBuilder: (context, error, stackTrace) {
            // 返回一个新的部件替换原有图片
            return Image.asset(
              "lib/assets/home_cmd_inner.png",
              // width: 100,
              height: 140,
              fit: BoxFit.cover,
            ); // Image.asset
          },
          list[index].picture,
          // width: 100,
          height: 140,
          fit: BoxFit.cover
```

运行flutter项目到安卓端-5.解决UI溢出

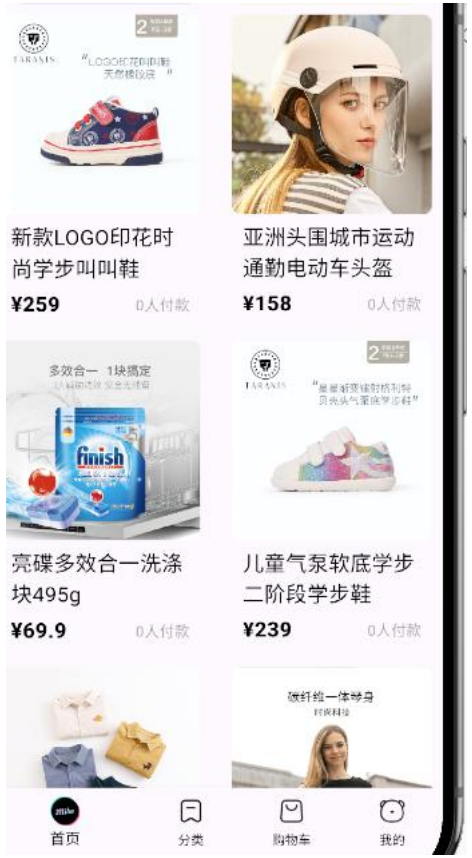
2.HmHot


```
// 构建子项
List<Widget> _getChildrenList() {
  return _items.map((item) {
    return Expanded(
      child: Container(
        decoration: BoxDecoration(borderRadius: BorderRadius.circular(12)),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ClipRRect(
              borderRadius: BorderRadius.circular(12),
              child: Image.network(
                item.picture,
                height: 100,
                fit: BoxFit.cover,
                errorBuilder: (context, error, stackTrace) {
                  return Image.asset(
                    "lib/assets/home_cmd_inner.png",
                    height: 100,
                  ); // Image.asset
                },
              ), // Image.network
            ), // ClipRRect
            SizedBox(height: 5),
```

```
child: Column(
  children: [
    // 顶部内容
    _buildHeader(),
    SizedBox(height: 10),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      spacing: 10,
      children: _getChildrenList(),
    ), // Row
  ],
), // Column
```

运行flutter项目到安卓端-5.解决UI溢出

3.HmMoreList



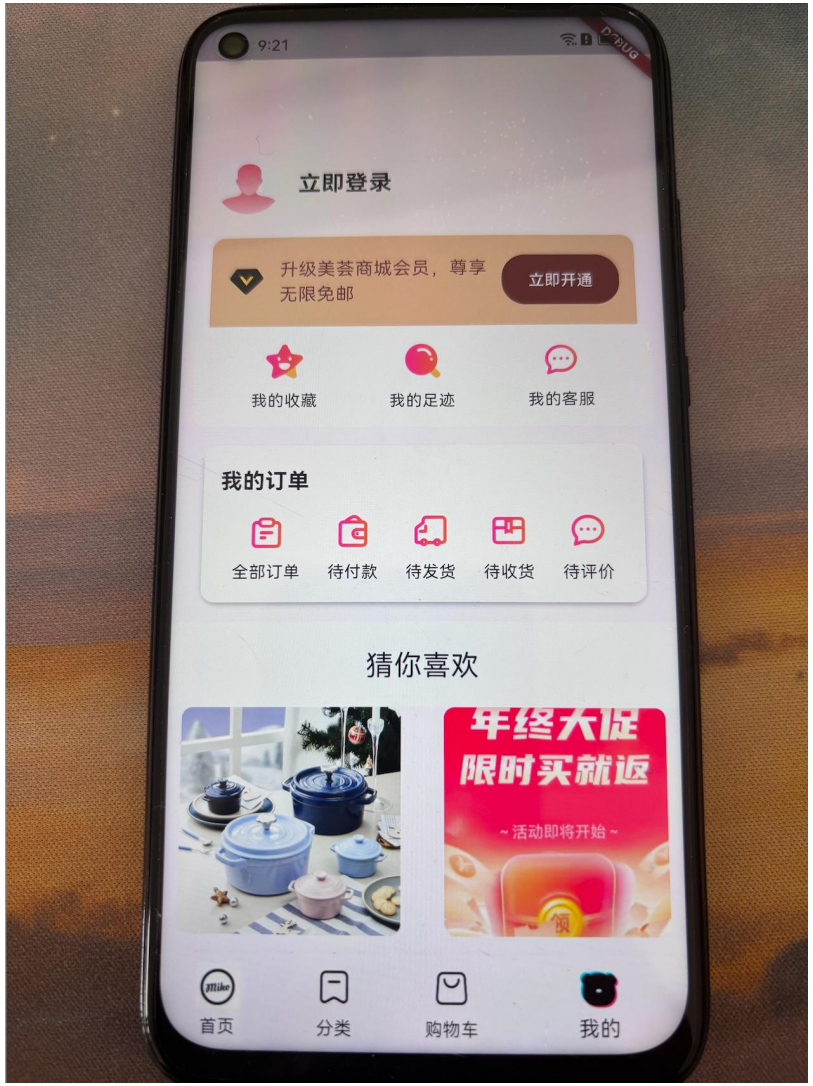

```
    // 网格是两列
    SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: 2,
      mainAxisSpacing: 10,
      crossAxisSpacing: 10,
      childAspectRatio: 0.7,
    ), // SliverGridDelegateWithFixedCrossAxisCount
itemBuilder: (BuildContext context, int index) {
  return Padding(
    padding: EdgeInsets.symmetric(horizontal: 10),
    child: _getChildren(index),      You, 上周 • 集成推荐列表
  ); // Padding
```

```
      fontWeight: FontWeight.w800,
    ), // TextStyle
    // children: [
    //   TextSpan(text: " "),
    //   TextSpan(
    //     text: "${widget.recommendList[index].price}",
    //     style: TextStyle(
    //       decoration: TextDecoration.lineThrough,
    //       color: Colors.grey,
    //       fontSize: 12,
    //     ),
    //   ),
    // ],
  ), // TextSpan
), // Text.rich
Text(
  "${widget.recommendList[index].payCount}人付款",
  style: TextStyle(color: Colors.grey),
```

运行flutter项目到安卓端-6.打包

```
flutter build apk --debug # 生成调试版 APK（未签名，用于测试）

flutter build apk --release # 生成发布版 APK（需要签名配置）
```

```
Try `flutter pub outdated` for more information.
Running Gradle task 'assembleDebug'...                             24.7s
✓ Built build/app/outputs/flutter-apk/app-debug.apk
gaolingyu@MacBook-Pro-4 hm_shop %
```



运行flutter项目到ios端(仅Mac端支持)

运行步骤

1. 安装xcode
2. 打开模拟器
3. 运行到模拟器
4. 解决UI超出问题

运行flutter项目到鸿蒙端

运行步骤

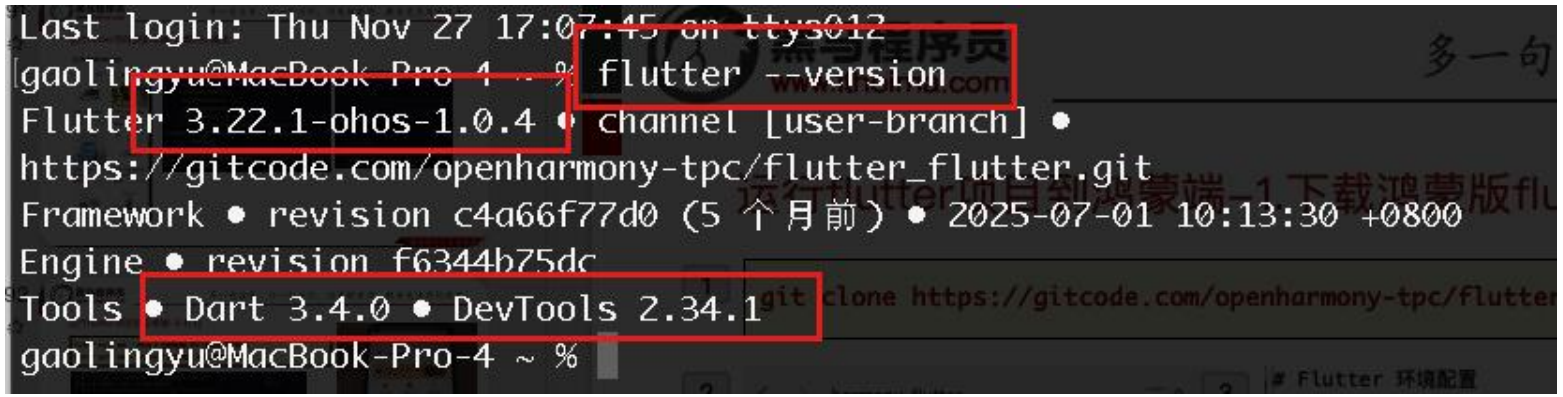
1. 下载鸿蒙版flutter替换原有版本
2. 安装DevEcoStudio-安装模拟器
3. 配置鸿蒙环境变量
4. 清理项目-降低版本-安装依赖
5. 项目签名运行到模拟器
6. 解决兼容性问题

运行flutter项目到鸿蒙端–1.下载鸿蒙版flutter

```
git clone https://gitcode.com/openharmony-tpc/flutter_flutter.git
```

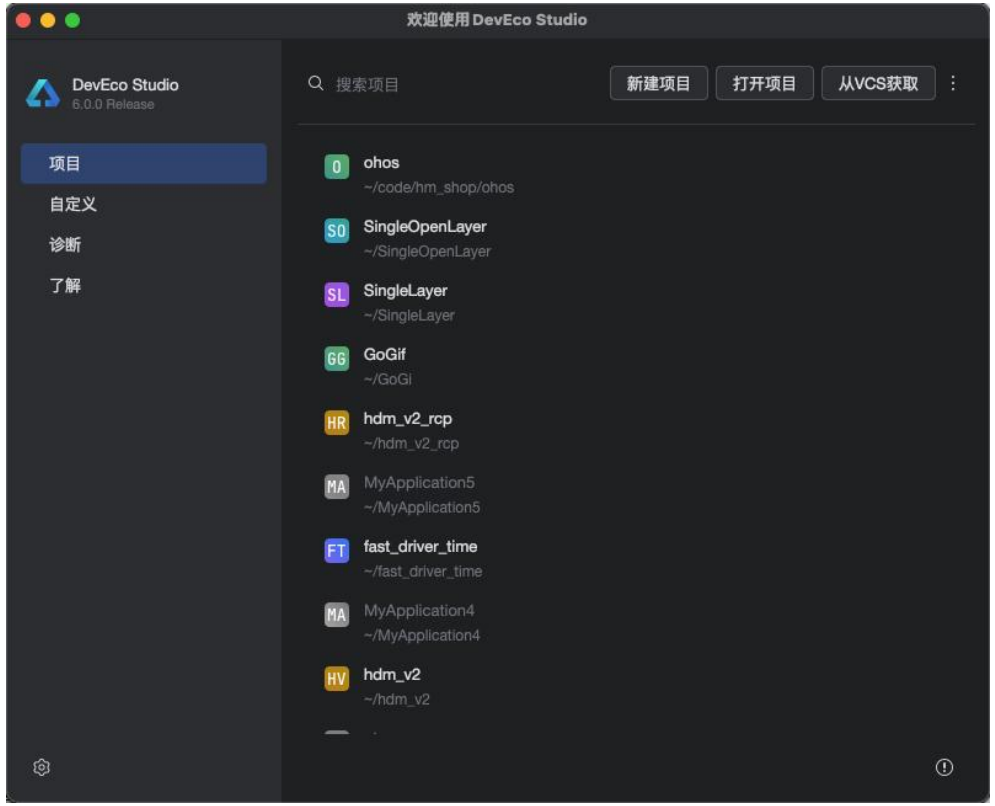

```
# Flutter 环境配置
 export PUB_HOSTED_URL="https://pub.flutter-io.cn"
 export FLUTTER_STORAGE_BASE_URL="https://storage.flutter-io.cn"
 export PATH="/Users/gaolingyu/harmony_flutter/bin:$PATH"
# export PATH="/Users/gaolingyu/flutter/bin:$PATH"
#export PATH="/Users/gaolingyu/flutter_flutter/bin:$PATH"

# harmony_flutter
```

替换原有的flutter的环境变量



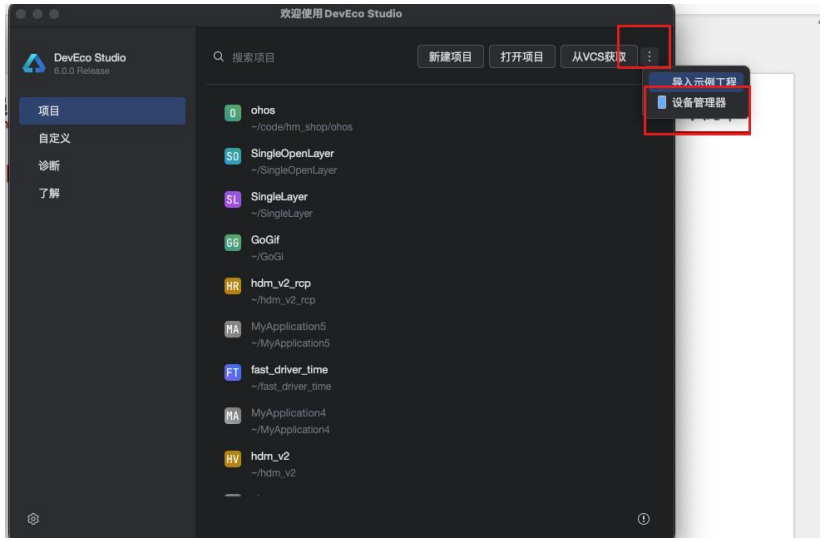
运行flutter项目到鸿蒙端–2. 安装DevEcoStudio

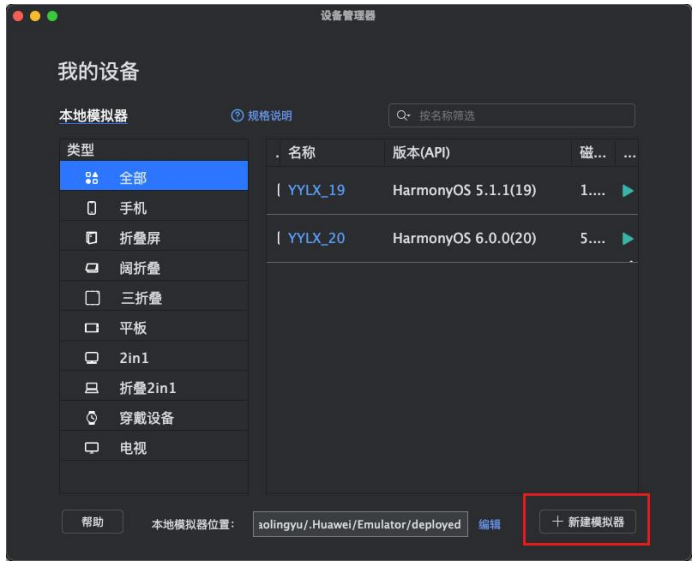
下载地址：https://developer.huawei.com/consumer/cn/download/

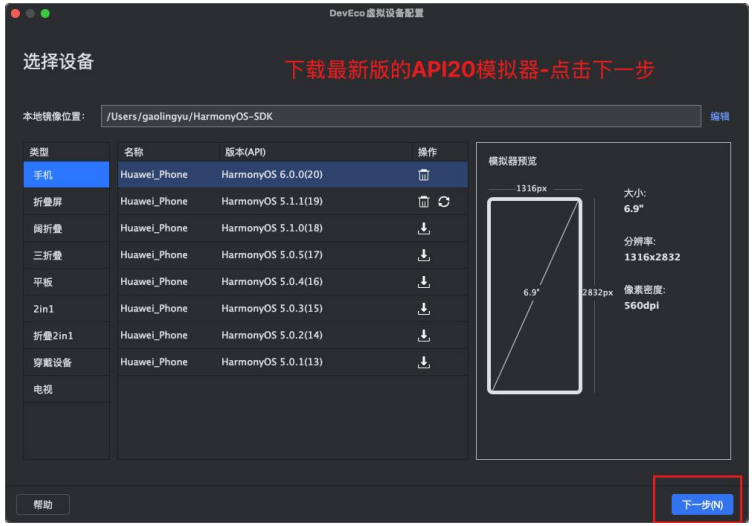


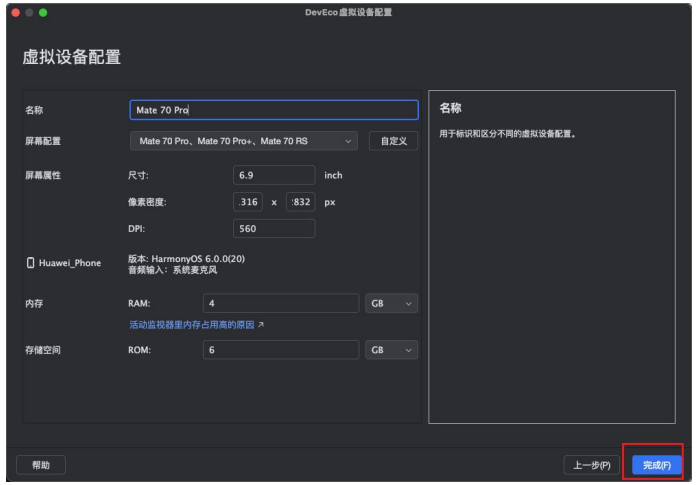
windows/mac-直接安装即可

运行flutter项目到鸿蒙端–2. 安装模拟器







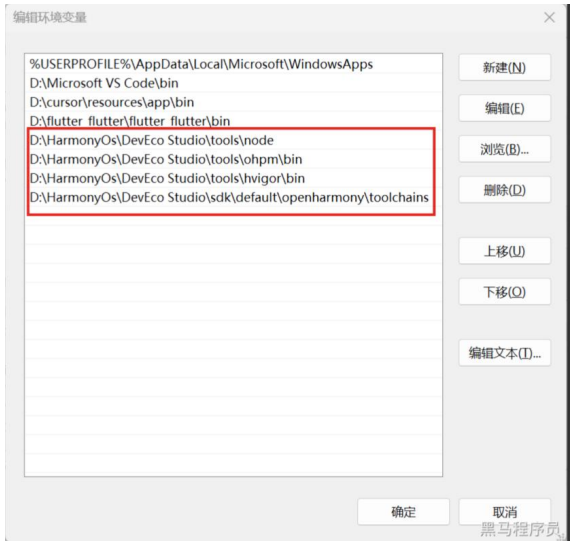


运行flutter项目到鸿蒙端-3. 配置鸿蒙环境变量

mac版(写入.zshrc或者.bash_profile)

```
export TOOL_HOME=/Applications/DevEco-Studio.app/Contents # mac环境
export DEVECO_SDK_HOME=$TOOL_HOME/sdk # sdk
export PATH=$TOOL_HOME/tools/ohpm/bin:$PATH # ohpm
export PATH=$TOOL_HOME/tools/hvigor/bin:$PATH # hvigor
export PATH=$TOOL_HOME/tools/node/bin:$PATH # node
export HDC_HOME=$TOOL_HOME/sdk/default/openharmony/toolchains # hdc
```

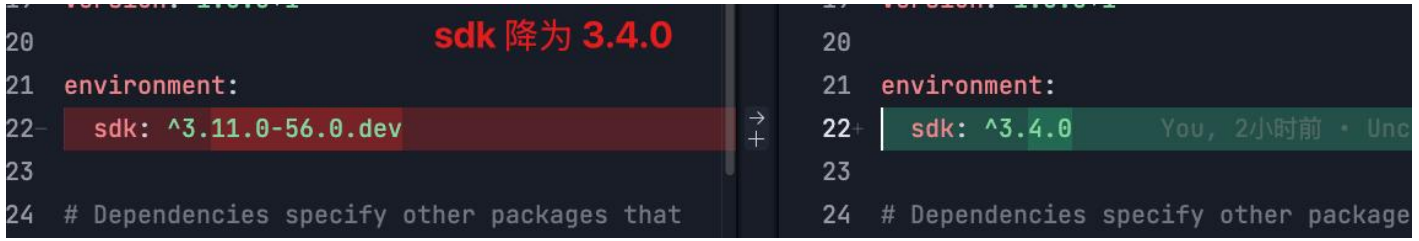
windows



检测环境： flutter doctor -v

运行flutter项目到鸿蒙端–3. 清理项目–降低版本–安装依赖

鸿蒙版flutter相较于最新版落后，需要降低使用版本




```
flutter clean && flutter pub get
```

依赖需要清理后重新安装

运行flutter项目到鸿蒙端–4.运行到模拟器(签名)

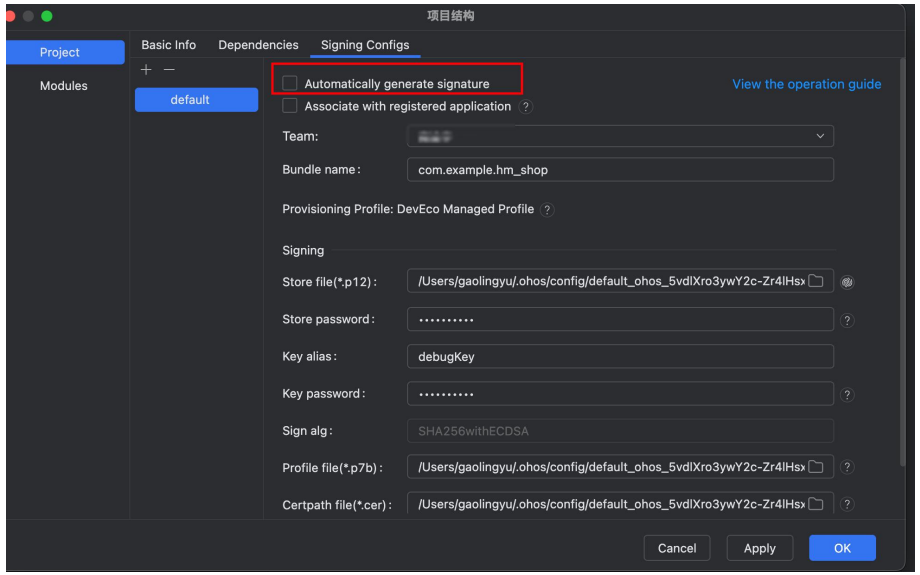
补全鸿蒙侧代码

```
flutter create .
```

打开鸿蒙模拟器-在项目中执行命令

```
flutter run # 选择对应的设备
```

使用DevEcoStudio对ohos项目进行签名



解决高版本语法问题

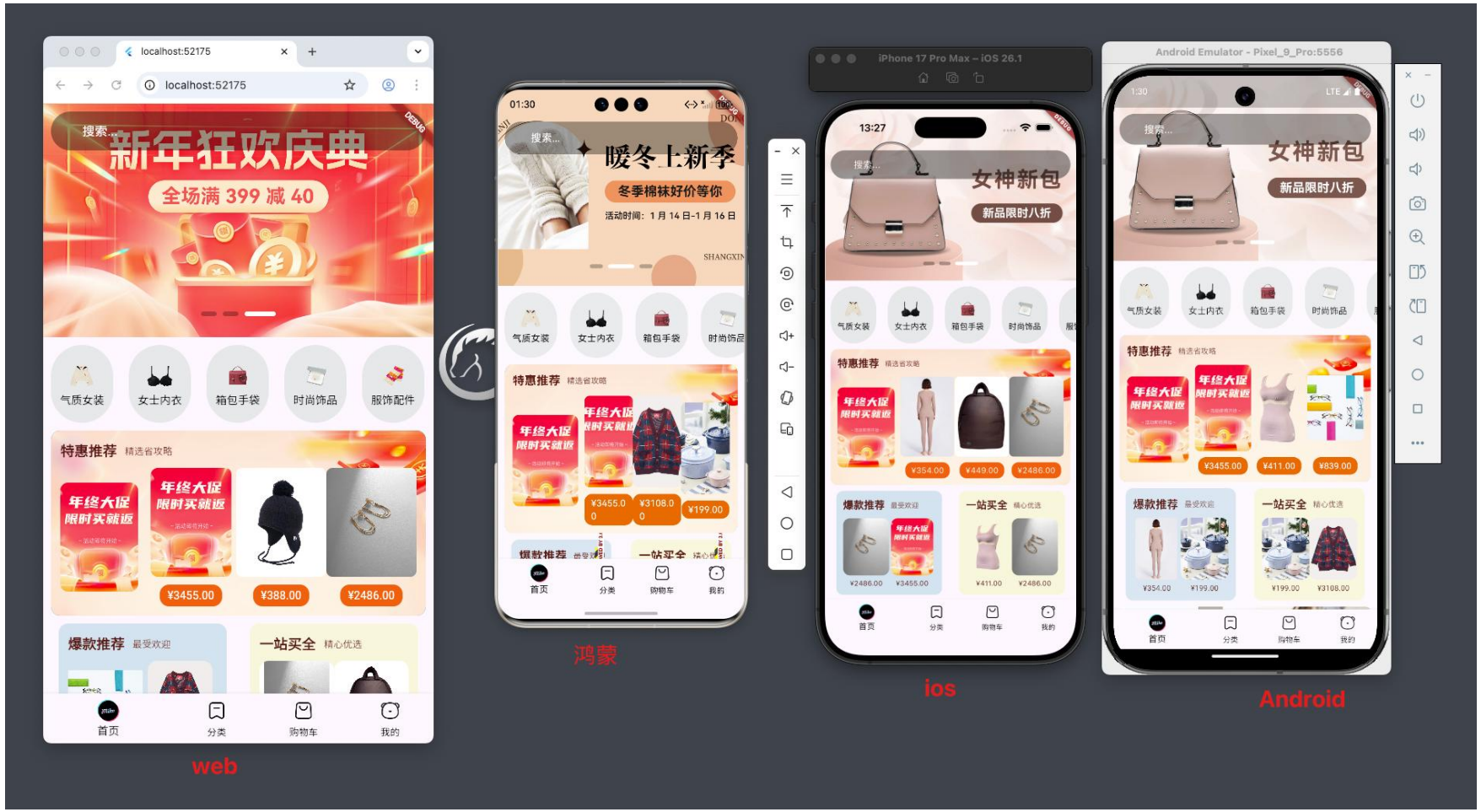
去掉spacing属性和圆角方法使用方式

运行flutter项目到鸿蒙端-5.解决兼容性问题

shared_preferences换成git仓库模式

```
shared_preferences:
  git:
    url: "https://gitcode.com/openharmony-sig/flutter_packages.git"
    path: "packages/shared_preferences/shared_preferences"
```

完结篇


localhost:52175
新年狂欢庆典
全场满 399 减 40
特惠推荐
爆款推荐
一站买全
web
暖冬上新季
冬季棉袜好价等你
鸿蒙
iPhone 17 Pro Max – iOS 26.1
女神新包
新品限时八折
ios
Android Emulator - Pixel_9_Pro:5556
Android

完结篇–安卓版本适配

由于之前Flutter-SDK的降级，需要针对安卓进行版本的设置

1.将 Android Gradle Plugin 从 7.3.0 升级到 8.3.1

2.将 Kotlin 版本从 1.7.10 升级到 1.9.22

3.将 Java 版本从 1.8 升级到 17

4.明确设置 NDK 版本为 25.1.8937393

5.删除了 settings.gradle.kts（已有 settings.gradle）

6.删除了 build.gradle.kts 文件（已有 build.gradle）

完结篇–安卓版本适配

android/app/build.gradle

```
android {
    namespace = "com.example.hm_shop"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = "25.1.8937393"

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }
```

android/settings.gradle

```
plugins {
    id "dev.flutter.flutter-plugin-loader" version "1.0.0"
    id "com.android.application" version "8.3.1" apply false
    id "org.jetbrains.kotlin.android" version "1.9.22" apply false
}

include ":app"
```

完结篇–感谢

感谢各位聆听的小伙伴

拥抱技术的变化，相信美好即将发生

莫道桑榆晚，为霞尚满天

课程结束